Simulating Spectral Sampling of the MICA Files with Enfys: InGaAs MWIR

August 15, 2025

1 Abstract

Here we investigate the expected ability of a new remote-sensing stand-off infrared spectrometer, *Enfys*, for the ExoMars Rosalind Franklin Rover, to resolve diagnostic features of the reflectance spectra of key minerals previously identified on the surface of Mars.

In this notebook we simulate Enfys performance using an InAs sensor, that is subject to thermal noise from the Enfys structure.

2 Introduction

As discussed in previous notebooks, Enfys is an Infra-Red point-spectrometer for the ExoMars Rosalind Franklin rover.

In these notebooks we explore how we expect Enfys to sample minerals of interest on the Mars surface.

In this notebook, we consider the expected performance of Enfys using an InGaAs Mid-Wave Infrared sensor for the wavelength range of 1.6 - 2.5 µm, replacing the InAs 1.6 - 3.1 µm sensor.

Included in the modelling is the noise expected for the sensor, as provided by the EXM-EN-MTX-ABU-0001_Enfys_Science_Traceability_Matrix_3-0 document.

2.1 Problem

This optical design is functionally different to the Acousto-Optic Tunable Filter design of ISEM, so it is important that the design is demonstrated to fulfil the performance requirements of the instrument, and that expectations of performance can be set prior to laboratory tests of the completed instrument.

At time of writing, the spectrometer is in the development stage. This notebook describes how the Spectral Parameters Toolkit[1] can be used to convert the developing expectations of the spectral range and resolution and radiometric sensitivity of the instrument into predictions of its ability to discriminate mineral species and phases on the Mars surface at the rover landing site.

[1]Stabbins, R., & Grindrod, P. (2024). The Spectral Parameters Toolkit (sptk): Release v0.1 (Version v0.1) [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.10694286

3 Method

To investigate the expected performance we use high-resolution laboratory measured spectra of a range of minerals expected at the Mars surface, and computationally simulate the sampling of the spectra with the expected spectral response of *Enfys*. We will perform repeat sampling with synthetically added noise to investigate the spectral and radiometric limits of the instrument.

We will produce a number of visualisations to investigate the predicted instrument performance, and demonstrate the limits of the instrument discriminatory ability.

3.1 Spectral Resampling

We take a simple approach to resampling, as follows.

We neglect reflectance calibration uncertainty, and any systematic uncertainty when merging the data from the two separate photodiodes, and assume that the linear-variable filter movement is perfectly calibrated such that a measurement at centre-wavelength λ_{cwl} can be requested with arbitrary precision.

We assume an observation of λ_{cwl} has a Cauchy transmission profile, and, prior to component characterisation, a spectral resolving power of $\lambda/\Delta\lambda = 100$ is achieved for all λ_{cwl}, such that the Cauchy transmission profile for λ_{cwl} has a full-width-at-half-maximum of $\Delta\lambda = \lambda_{cwl}/100$.

The Cauchy transmission profile $T(\lambda|\lambda_{cwl})$ is given by

$$T(\lambda|\lambda_{cwl}) = \frac{\gamma^2}{(\lambda - \lambda_{cwl}^2) + \gamma^2}$$

where $\gamma = \Delta\lambda/2 = (\lambda_{cwl}/100)/2$.

We assume that a high-resolution laboratory spectrum approximates the continuous function $R(\lambda)$, such that the calibrated reflectance at λ_{cwl} has the expected value $R[\lambda_{cwl}]$ of:

$$R[\lambda_{cwl}] = \frac{\int R(\lambda)T(\lambda|\lambda_{cwl})d\lambda}{\int T(\lambda|\lambda_{cwl})d\lambda}$$

To simulate noise we add ε, an additive noise term that we draw from the Gaussian distribution with variance:

$$\sigma^2 = \frac{\sqrt{R[\lambda_{cwl}]}}{\text{SNR}}$$

under the assumption that maximum SNR is only achieved for $R = 1$, and the noise is dominated by counting noise, such that σ follows a Poissonian relationship with the signal R.

For each observation $R[\lambda_{cwl}]$ we draw 100 repeat samples, such that

$$R_i[\lambda_{cwl}] = R[\lambda_{cwl}] + \varepsilon_i$$

with

$$\varepsilon_i \sim \mathcal{N}\left(0, \frac{\sqrt{R[\lambda_{cwl}]}}{\text{SNR}}\right)$$

This gives an indication of the expected noise of a calibrated *Enfys* acquired spectrum.

3.2 Analysis

For now, we simply produce plots of the resampled spectra, and visually inspect the plots to consider the ability to resolve diagnostic spectral features against the noise. We will include more details of analysis as that is developed here.

3.3 Notebook and Environment Setup

Here we include the required code blocks for setting up this notebook for this study.

[1]:
```
%load_ext autoreload
%autoreload 2
%config InlineBackend.figure_format = 'svg' # note - requires Inkscape to be␣
 ↪installed, and to be accessible from the terminal
# on macos, add inkscape to the path with `sudo ln -s /Applications/Inkscape.
 ↪app/Contents/MacOS/inkscape /usr/local/bin/inkscape`
%matplotlib inline
```

We will be using the Spectral Parameters Toolkit. First we import the required modules.

[2]:
```
import sptk.config as cfg
from sptk.material_collection import MaterialCollection
from sptk.instrument import Instrument
from sptk.observation import Observation
from sptk.spectral_library_analyser import SpectralLibraryAnalyser as sla
```

We set our simulation resolution range to extend the range of Enfys, of 0.9 - 3.1 µm, such that the long tails of the Cauchy distribution of the linear-variable-filter profiles can be accommodated during sampling. We leave our spectral resolution at 1 nm for all λ.

[3]:
```
cfg.update_sample_res('wvl_min', 380)
cfg.update_sample_res('wvl_max', 3300)
```

Finally we set the project name.

[4]:
```
PROJECT_NAME = 'enfys_mica_ingaas_mwir'
```

4 Data

For this study we are using a spectral library of minerals identified through CRISM analysis (the MICA files)[2]. Here we use the laboratory spectra that have been identified that best represent the

[2]Viviano-Beck, C. E., Seelos, F. P., Murchie, S. L., Kahn, E. G., Seelos, K. D., Taylor, H. W., Taylor, K., Ehlmann, B. L., Wiseman, S. M., Mustard, J. F., & Morgan, M. F. (2014). Revised CRISM spectral parameters and summary

type localities presented in the MICA files. These can be accessed here, and are sourced from the RELAB and USGS public spectral libraries.

We have grouped the spectral library into categories as follows, and included this categorisation in the sptk distribution, accessed from the `sptk.material_sets` module.

[5]:
```
from sptk.material_sets import MICA_SET
```

We use the `sptk.material_collection` module to parse the spectral library:

[6]:
```
matcol = MaterialCollection(
        MICA_SET,
        'MICA_new_dirtree',
        PROJECT_NAME,
        load_existing=False,
        balance_classes=False,
        random_bias_seed=None,
        allow_out_of_bounds=True,
        plot_profiles=False,
        export_df=True)
```

```
Building new enfys_mica_ingaas_mwir MaterialCollection DF

---------------------------------
Category
  species|subgroup|group|
     data_id status
---------------------------------
Iron Oxides & Primary Silicates
  -clinopyroxene|pyroxenes|inosilicates|
     -NBPP22.csv loaded
  -orthopyroxene|pyroxenes|inosilicates|
     -LAPP47B.csv loaded
  -fayalite|olivines|nesosilicates|
     -LAPO05.csv loaded
  -forsterite|olivines|nesosilicates|
     -LASC02.csv loaded
  -hematite|hematites|oxides|
     -GDS27.csv loaded
  -plagioclase|feldspars|tectosilicates|
     -NCLS04.csv loaded
---------------------------------
Sulfates
  -alunite|hydrous_sulphates|sulphates|
     -F1CC08B.csv loaded
  -bassanite|hydrous_sulphates|sulphates|
     -GDS145.csv loaded
```

products based on the currently detected mineral diversity on Mars. Journal of Geophysical Research: Planets, 119(6), 1403–1431. https://doi.org/10.1002/2014JE004627

```
  -gypsum|hydrous_sulphates|sulphates|
     -LASF41A.csv loaded
  -jarosite|hydrous_sulphates|sulphates|
     -LASF21A.csv loaded
  -kieserite|hydrous_sulphates|sulphates|
     -F1CC15.csv loaded
  -mg-sulphate|hydrous_sulphates|sulphates|
     -799F366.csv loaded
---------------------------------
Phyllosilicates
  -chlorite|chlorites|phyllosilicates|
     -LACL14.csv loaded
  -kaolinite|kaolinites|phyllosilicates|
     -LAKA04.csv loaded
  -illite|micas|phyllosilicates|
     -LAIL02.csv loaded
  -margarite|micas|phyllosilicates|
     -GDS106.csv loaded
  -vermiculite|micas|phyllosilicates|
     -c1cy29.csv loaded
  -serpentine|serpentines|phyllosilicates|
     -LALZ01.csv loaded
  -montmorillonite|smectites|phyllosilicates|
     -LAMO02.csv loaded
  -nontronite|smectites|phyllosilicates|
     -NCJB26.csv loaded
  -saponite|smectites|phyllosilicates|
     -LASA58.csv loaded
  -talc|talcs|phyllosilicates|
     -GDS23.csv loaded
---------------------------------
Carbonates
  -calcite|calcites|carbonates|
     -CAGR01.csv loaded
  -magnesite|calcites|carbonates|
     -F1CC06B.csv loaded
---------------------------------
Hydrated Silicates & Halides
  -halite|chlorides|halides|
     -HS433.csv loaded
  -prehnite|prehnites|inosilicates|
     -LAZE03.csv loaded
  -siliceous-sinter|hydrated-silicas|mineraloids|
     -bkr1jb329c.csv loaded
  -epidote|epidotes|sorosilicates|
     -GDS26.csv loaded
  -analcime|zeolites|tectosilicates|
     -GDS1.csv loaded
```

```
Loading 29 of entries complete.
Exporting the Material Collection to CSV and Pickle formats…
```

We can use the `material_collection` plotting routines to visualise the library.

[7]:
```
ax = matcol.plot_profiles(stacked=True, scope='categories', groupby='Species')
```

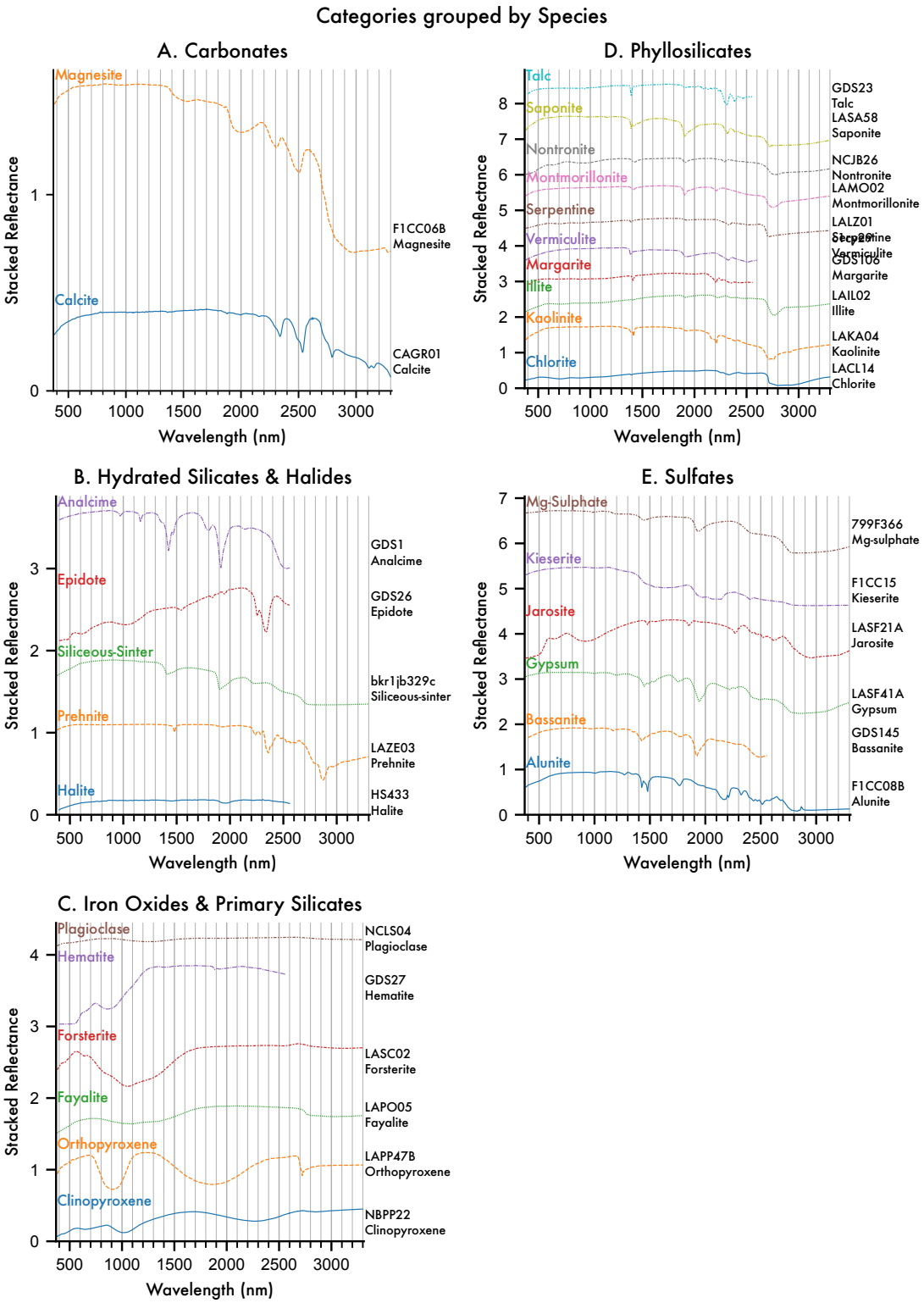


Figure 2 High resolution MICA Files spectral library, arranged by category and grouped by mineral

species.

We can see that not all of the entries cover the complete 0.9 - 3.1 µm range of *Enfys*, so in future work we should look to replace these MICA files laboratory type spectra with spectra that all span the entire range.

We can also produce a waterfall visualisation, an alternative plotting method that presents the information in a condensed format, that is useful for highlighting where absorption features lie.

[8]:
```
axes = matcol.plot_profiles(waterfall=True, scope='all', groupby='Category')
```

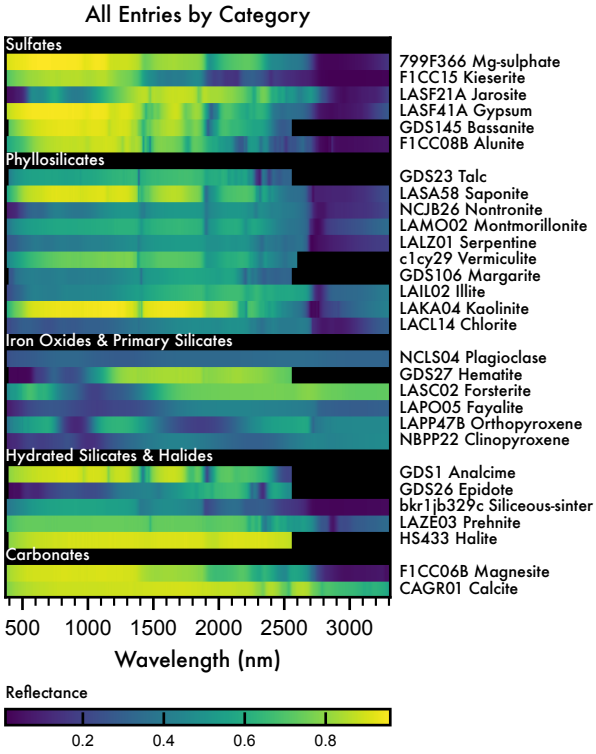


Figure 3 Waterfall plot of the high resolution MICA Files spectral library, arranged by category and grouped by mineral species.

A more useful version of this plot format is to show band locations after continuum removal. We can use the `SpectralLibraryAnalyser.remove_continuum()` function to produce a duplicate of the `MaterialCollection` but with the data continuum-removed.

[9]:
```
from sptk.spectral_library_analyser import SpectralLibraryAnalyser as sla

matcol_sla = sla(matcol)
matcol_cr = matcol_sla.remove_continuum()
```

We can plot the profiles to illustrate the continuum removal. Here we use the `scope = 'all'` version of the profile plot.

[10]:
```
axes = matcol_cr.plot_profiles(stacked=True, scope='all', groupby='Category')
```

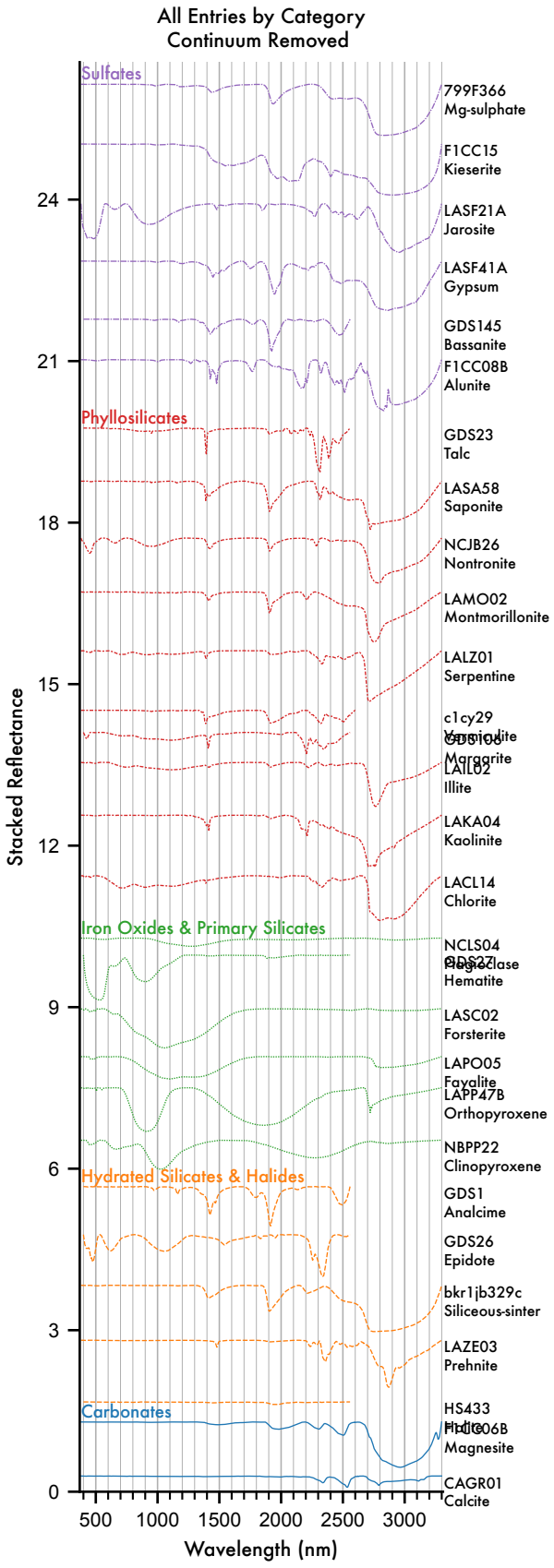

All Entries by Category
Continuum Removed
Sulfates
Phyllosilicates
Iron Oxides & Primary Silicates
Hydrated Silicates & Halides
Carbonates
799F366
Mg-sulphate
F1CC15
Kieserite
LASF21A
Jarosite
LASF41A
Gypsum
GDS145
Bassanite
F1CC08B
Alunite
GDS23
Talc
LASA58
Saponite
NCJB26
Nontronite
LAMO02
Montmorillonite
LALZ01
Serpentine
c1cy29
Vermiculite
GDS105
Margarite
LAIL02
Illite
LAKA04
Kaolinite
LACL14
Chlorite
NCLS04
Hematite
LASC02
Forsterite
LAPO05
Fayalite
LAPP47B
Orthopyroxene
NBPP22
Clinopyroxene
GDS1
Analcime
GDS26
Epidote
bkr1jb329c
Siliceous-sinter
LAZE03
Prehnite
HS433
Magnesite
CAGR01
Calcite
Stacked Reflectance
0
3
6
9
12
15
18
21
24
500
1000
1500
2000
2500
3000
Wavelength (nm)

Figure 4 Continuum-removed high resolution MICA Files spectral library, grouped by Category.

Now we plot the waterfall representation:

[11]:
```
axes = matcol_cr.plot_profiles(waterfall=True, scope='all', groupby='Category')
```

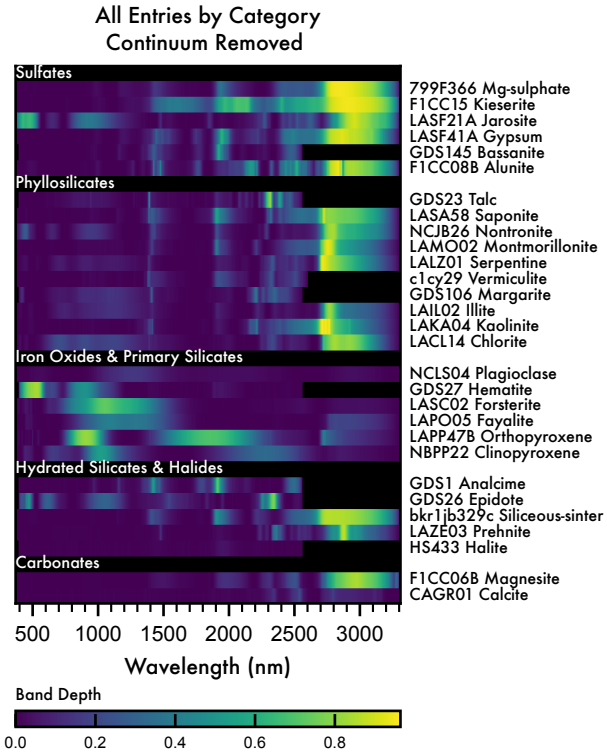


Figure 5 Continuum-removed waterfall plot of high resolution MICA Files spectral library, grouped by Category.

Again, we can see that the waterfall plot represents the band depth information of the `MaterialCollection` in a compact format. Scanning vertically allows for quick identification of common band locations, their depths, widths, and asymmetries.

5 Results

5.1 Simulating the Enfys Spectral Response

To simulate Enfys, rather than providing a file with the expected spectral response of each position of the Linear-Variable Filter (LVF, above), we specify the spectral range and resolution and sampling regime, and generate a set of central-wavelengths and bandwidths. We do this with the `InstrumentBuilder` class of the `sptk.instrument` module.

[12]:
```
from sptk.instrument import InstrumentBuilder
```

The expected spectral resolution of Enfys is $\lambda/\Delta\lambda = 100$, and the range when using the InGa sensor is 0.9 - 3.1 µm.

5.1.1 Signal-to-Noise Ratio

We obtain the expected SNR from the enfys_inas_mwir_snr.csv file, derived from the EXM-EN-MTX-ABU-0001_Enfys_Science_Traceability_Matrix_3-0 spreadsheet.

We have adapted the InstrumentBuilder function to accept a pandas Series mapping wavelength to SNR. The InstrumentBuilder then performs interpolation to the simulated wavelength range.

We read in the SNR file:

[13]:
```
import pandas as pd

enfys_snr = pd.read_csv('../data/instruments/enfys_snr.csv', index_col=0)
```

and get the SNR data for the InAs MWIR sensor,

[14]:
```
ingaas_mwir_snr = enfys_snr['InGaAs MWIR SNR']
```

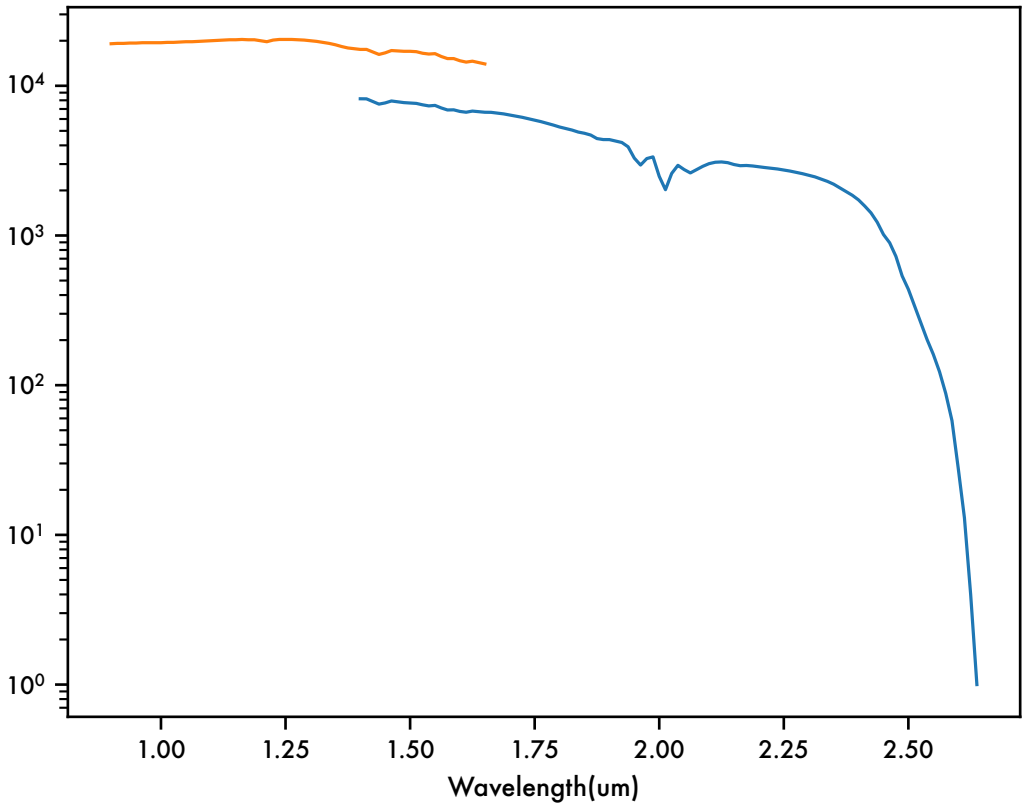
and the InGaAs SWIR sensor,

[15]:
```
ingaas_swir_snr = enfys_snr['InGaAs SWIR SNR']
```

and then we splice these together, using the higher value SNR where the wavelength ranges overlap:

[16]:
```
ingaas_mwir_snr.plot(logy=True)
ingaas_swir_snr.plot()
```

[16]:
```
<AxesSubplot: xlabel='Wavelength(um)'>
```



[17]:
```
# concatanate the inas_mwir_snr and the ingaas_swir, using the highest SNR for␣
 ↪overlapping wavelength values
enfys_ingaas_mwir_snr = ingaas_swir_snr.combine_first(ingaas_mwir_snr)
```

We also convert the wavelengths from µm to nm:

[18]:
```
enfys_ingaas_mwir_snr.index = enfys_ingaas_mwir_snr.index * 1000
```

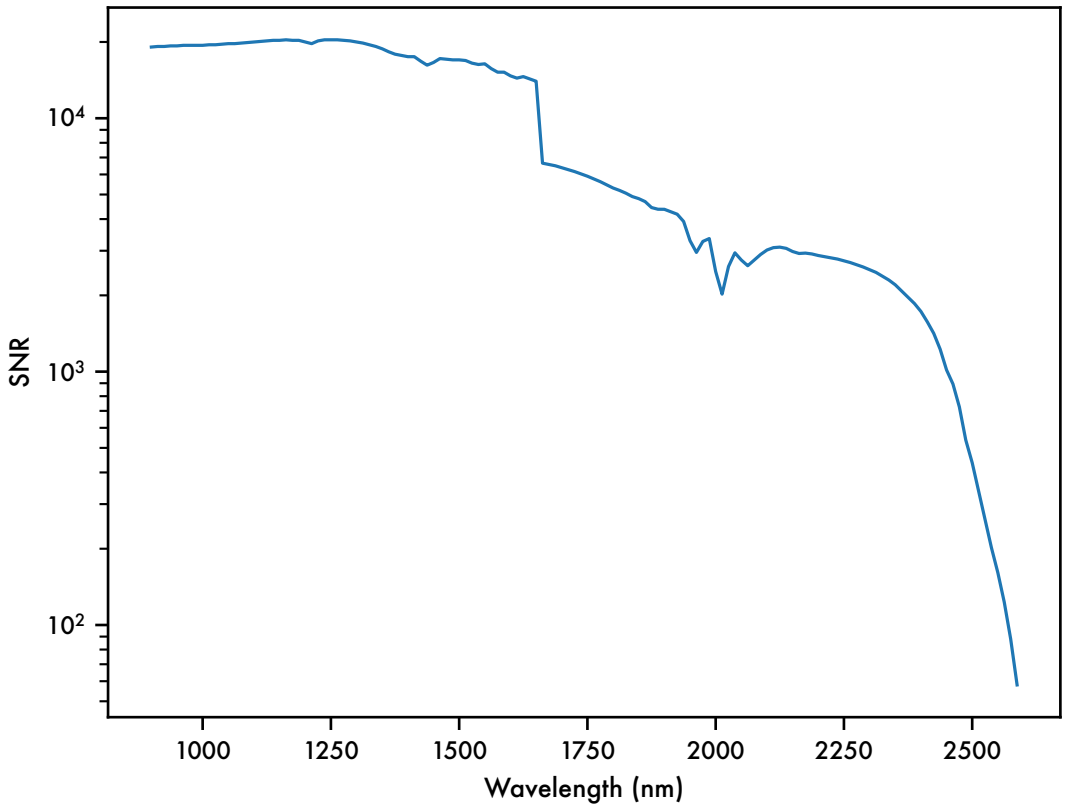
for computational convenience, we should set values of SNR<=1 to NaN, to interpret these as 'bad bands'

[19]:
```
# where SNR <=1 to NaN, to interpret these as 'bad bands'
import numpy as np
enfys_ingaas_mwir_snr = enfys_ingaas_mwir_snr.where(enfys_ingaas_mwir_snr > 30,␣
 ↪other=np.nan)
```

[20]:
```
enfys_ingaas_mwir_snr.index.name = 'wavelength'
```

[21]:
```
g = enfys_ingaas_mwir_snr.plot(logy=True)
# set the x and y axes
g.set_xlabel('Wavelength (nm)')
g.set_ylabel('SNR')
```

[21]: Text(0, 0.5, 'SNR')



5.1.2 Spectral Resolving Power

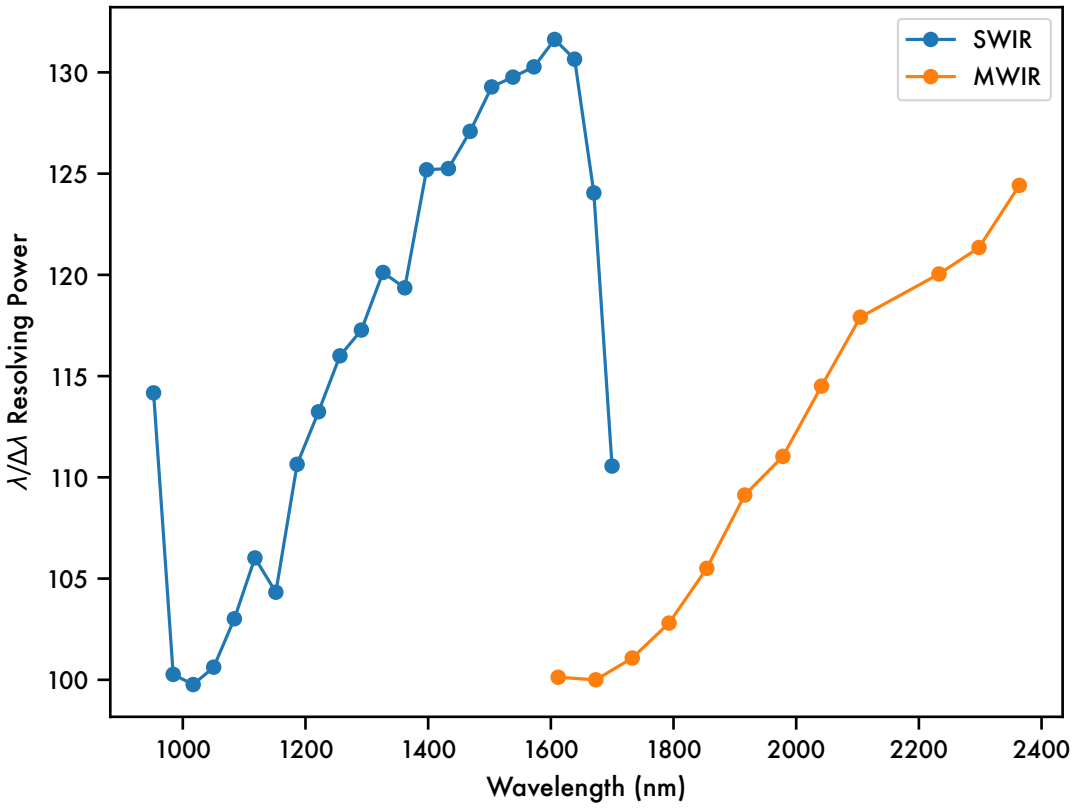
We can also load in new data for the spectral resolving power of the SWIR and MWIR channels of Enfys.

This data has been digitsed from the plot reported in the Enfys schedule review 20250807 ppt, by Matt Gunn.

[22]:
```
enfys_swir_respow = pd.read_csv('../data/instruments/enfys_swir_respow.csv',␣
↪index_col=0)
enfys_mwir_respow = pd.read_csv('../data/instruments/enfys_mwir_respow.csv',␣
↪index_col=0)
```

[23]:
```
g=enfys_swir_respow.plot(label='SWIR', marker='o')
enfys_mwir_respow.plot(marker='o',label='MWIR', xlabel='Wavelength (nm)',␣
↪ylabel=r'$\lambda/\Delta\lambda$ Resolving Power',ax=g)
g.legend(['SWIR', 'MWIR'])
```

[23]: <matplotlib.legend.Legend at 0x30beba410>



We now need to merge these datasets, following the same rule as the SNR merger, i.e. ensuring that the correct channel is used for the overlapping data.

[24]:
```
# get the max wavelength where ingaas_swir_snr is not nan
ingaas_swir_snr = ingaas_swir_snr[~np.isnan(ingaas_swir_snr)]
max_swir_wavelength = ingaas_swir_snr.index[-1] * 1000 # convert from µm to nm
max_swir_wavelength
```

[24]:
```
1650.0
```

[25]:
```
# get the enfys_swir_respow data where the wavelength is less than the␣
 ↪max_swir_wavelength
enfys_swir_respow = enfys_swir_respow[enfys_swir_respow.index <␣
 ↪max_swir_wavelength]
enfys_swir_respow
```

[25]:

Wavelength	Resolving Power
952.350	114.172
984.300	100.267
1016.886	99.765

```
1050.340            100.625
1084.064            103.015
1117.569            106.020
1151.685            104.326
1186.342            110.645
1221.214            113.238
1256.182            116.001
1291.127            117.274
1326.348            120.119
1361.845            119.359
1397.410            125.192
1432.961            125.249
1468.431            127.087
1503.552            129.283
1538.147            129.767
1572.489            130.274
1606.157            131.627
1638.722            130.653
```

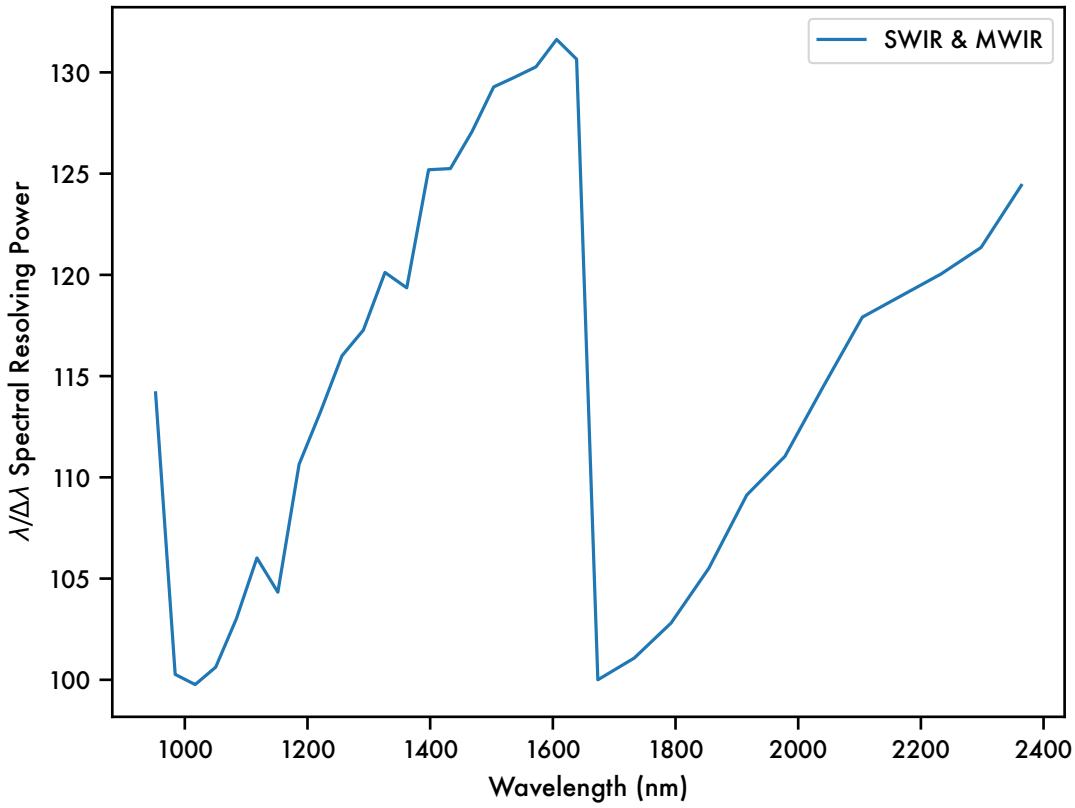
[26]:
```
# get the enfys_mwir_respow where the wavelength is greater than the␣
↪max_swir_wavelength
enfys_mwir_respow = enfys_mwir_respow[enfys_mwir_respow.index >␣
↪max_swir_wavelength]
enfys_mwir_respow
```

[26]:
```
            Resolving Power
Wavelength
1673.354            100.000
1732.701            101.075
1792.699            102.803
1854.283            105.507
1916.113            109.126
1978.328            111.035
2041.291            114.507
2104.575            117.912
2232.792            120.042
2298.070            121.352
2363.760            124.419
```

[27]:
```
# now merge the datasets
enfys_respow = pd.concat([enfys_swir_respow, enfys_mwir_respow])
```

[28]:
```
g=enfys_respow.plot(xlabel='Wavelength (nm)', ylabel=r'$\lambda/\Delta\lambda$␣
↪Spectral Resolving Power')
g.legend(['SWIR & MWIR'])
```

[28]:
```
<matplotlib.legend.Legend at 0x30bf55540>
```



We now need to pass this data to the InstrumentBuilder function.

5.1.3 Instrument Build

Now we will pass this to the InstrumentBuilder:

[29]:
```
enfys_builder = InstrumentBuilder(
    instrument_name='enfys_ingaas_mwir',
    instrument_type='lvf',
    sampling='nyquist',
    resolution = enfys_respow,
    spectral_range=[900, 2501],
    snr = enfys_ingaas_mwir_snr)
```

```
Exporting instrument to ../data/instruments/enfys_ingaas_mwir.csv…
```

[30]:
```
from sptk.instrument import Instrument
```

Now we build the transmission profiles for each LVF position, according to the list of CWLs and FWHMs generated above. These are accessed by reading the `.csv` file produced by the `InstrumentBuilder` procedure.

When generating the profiles, we turn the CWL and FWHM into a profile by assuming a distribution

function. For Enfys we use the Cauchy distribution, which behaves similarly to a Gaussian, but with wider 'wings'. It is parameterised by the full-width-at-half-maximum, as the distribution strictly has no finite variance. We have implemented a `build_cauchy_filter` method for the `Instrument` class.

[31]:
```
enfys = Instrument(
            name = 'enfys_ingaas_mwir', # filename of the instrument␣
↪information, held in /software/data/instruments/
            project_name = matcol.project_name, # the project directory name,␣
↪hosting outputs in the /spectral_parameter_studies/ directory
            shape = 'cauchy',
            load_existing=False, # if True, load existing instrument data from␣
↪the project directory
            plot_profiles=False, # if true, plot the transmission profiles␣
↪during construction
            export_df=True) # if True, export the instrument transmission␣
↪profiles to the project directory
```

```
Building Instrument…
Building new DataFrame for 'enfys_ingaas_mwir'…
Exporting the Instrument to CSV and Pickle formats…
```

[32]:
```
fig ,ax = enfys.plot_instrument_characteristics()
```

```
Plotting Instrument Transmission…

/opt/homebrew/Caskroom/miniconda/base/envs/enfys_sptk/lib/python3.10/site-
packages/seaborn/_oldcore.py:200: FutureWarning: elementwise comparison failed;
returning scalar instead, but in the future will perform elementwise comparison
  if palette in QUAL_PALETTES:

Plotting Maximum Signal-to-Noise Ratio…
Plotting FWHM…
Plotting Spectral Resolution…
Plots exported to ../projects/enfys_mica_ingaas_mwir/instrument
```

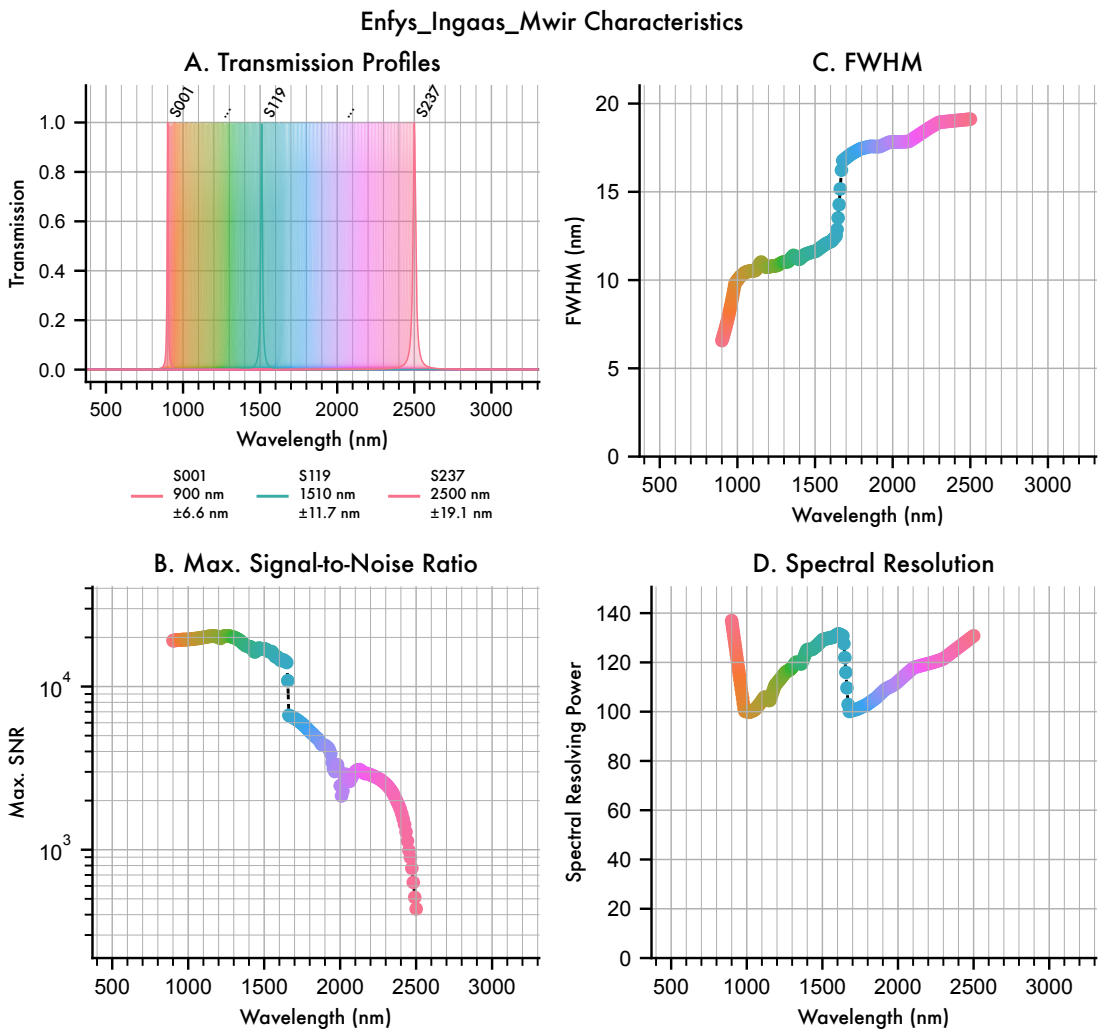


Figure 6 Simulated spectral response and resolution of *Enfys*, based on preliminary expectations of the spectral and radiometric response.

The data shown in figure 3 approximates the expected performance of *Enfys*. Once the instrument is calibrated at the component and assembled levels, this data can be replaced with empirically aqcuired data.

5.2 Sampling MICA with Enfys

We now make an `Observation` object by sampling the MICA files `MaterialCollection` with the Enfys `Instrument`.

[33]:
```
from sptk.observation import Observation
```

[34]:
```
obs = Observation(
    material_collection=matcol,
```

```
    instrument = enfys,
    load_existing=False,
    plot_profiles=False,
    export_df=True)
```

```
Building new Observation DataFrame
for 'enfys_mica_ingaas_mwir'…
Sampling the Material Collection with the Instrument…
Sampling complete.
Exporting the Observation Pickle format…
Observation export complete.
```

We can plot the sampled spectra against the input high-spectra below:

[35]:
```
axes = obs.plot_profiles(scope='categories', groupby='Species', stacked=True,␣
↪hires_under=True)
```

```
Negative level values detected
Negative level values detected
```

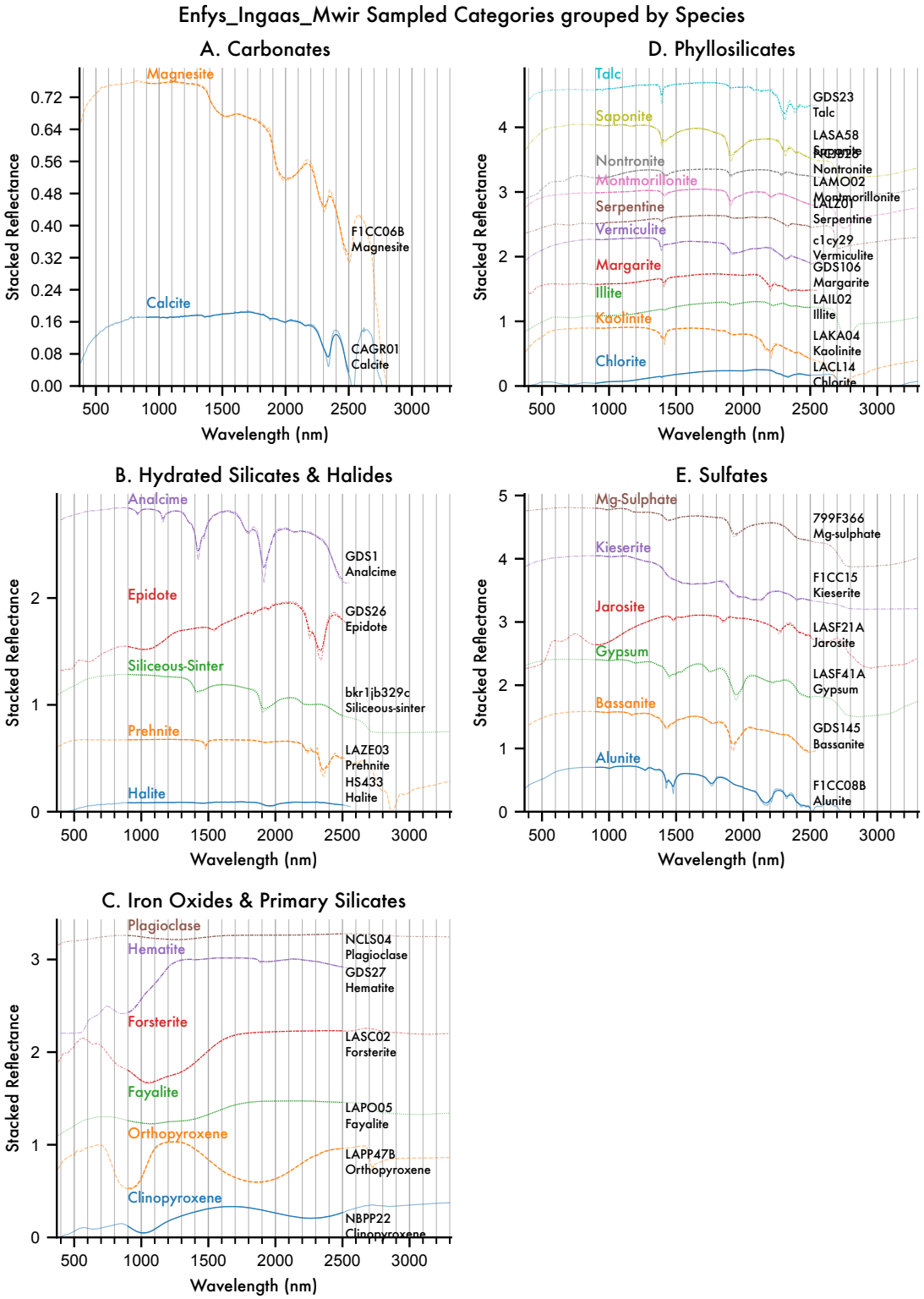


Figure 7 The MICA files as sampled by *Enfys*, with the original high-resolution spectra plotted

underneath each.

We can now see where there are deviations of the *Enfys* sampled spectra from the high-resolution source spectra.

For example, illite, plotted below for clarity.

[36]:
```
axes = obs.plot_profiles(scope='illite', groupby='Sample ID', stacked=False,␣
↪hires_under=True)
```

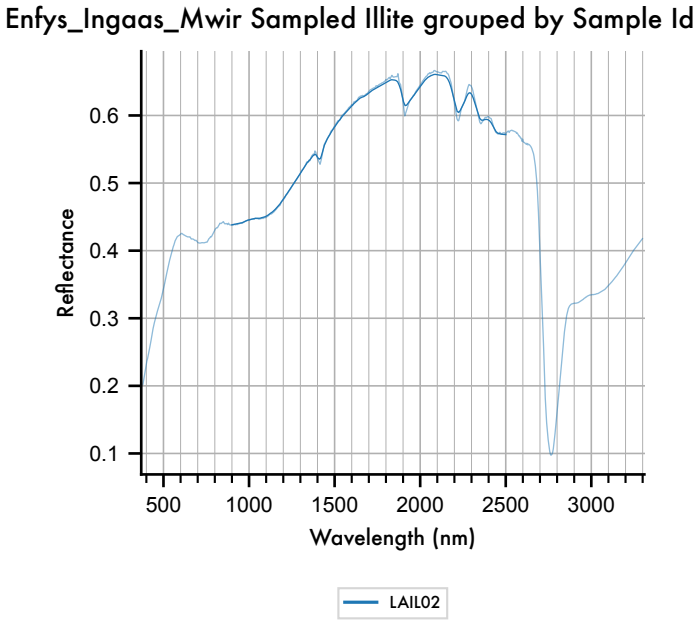


Figure 8 Deviation of the *Enfys* sampled spectrum from the high-resolution input spectrum for Illite.

We note that there is an effect by which peaks and troughs in the spectra are reduced. This is due to the long tails of the Cauchy filter profiles. Despite the reduction of absolute band depth, the features of Illite (fig. 8) are still resolvable.

We can produce a continuum-removed waterfall plot to check that the major bands are resolved.

[37]:
```
from sptk.spectral_library_analyser import SpectralLibraryAnalyser as sla

obs_cr = sla(obs).remove_continuum()
```

[38]:
```
axes = obs_cr.plot_profiles(waterfall=True, scope='all', groupby='Category')
```

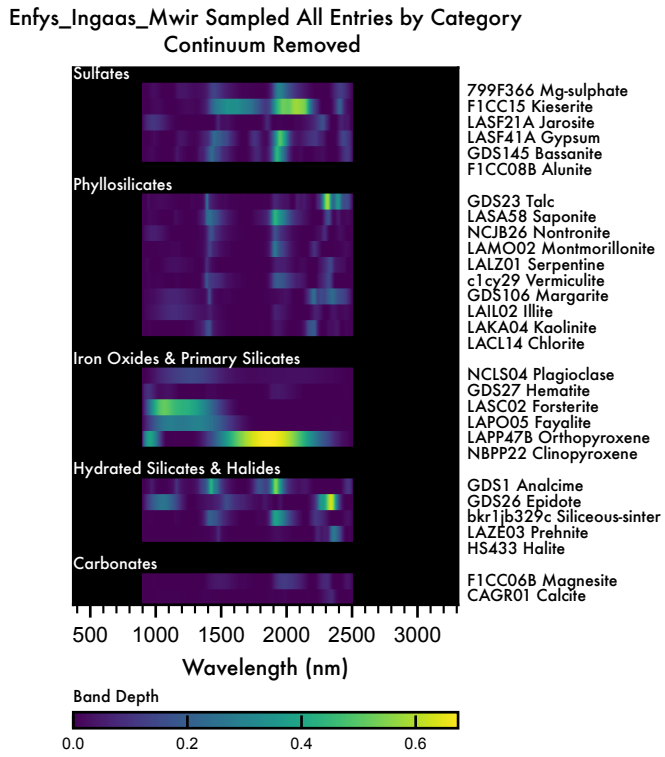


Figure 9 Continuum-Removed waterfall plot of the MICA files as sampled by *Enfys*.

Comparing Figure 9 to Figure 5, we can see that the weaker sub-2 µm bands are less well resolved.

We can save this data

[39]:
```
obs_cr.export_main_df()
```

```
Exporting the Observation Pickle format…
Observation export complete.
```

5.3 Adding Synthetic Noise

We have the capability to add synthetic noise to the dataset, by redrawing a sample from the dataset with noise added according to the signal-to-noise ratio specified for the given channel.

We can optionally redraw multiple noisy samples for each entry, to give an indication of the statistical uncertainty, or for example draw a single noisy sample, to produce a single-observation mission-realistic dataset.

For an entry with reflectance $R[\lambda_{cwl}]$, we redraw a sample:

$\tilde{R}_i[\lambda_{cwl}] = R[\lambda_{cwl}] + \epsilon_i$

where ϵ_i is drawn from:

$\epsilon_i \sim \mathcal{N}(0, \sigma[\lambda_{cwl}]^2)$

where $\sigma[\lambda_{cwl}]$ is the noise on the reflectance measurement, given by:

$\sigma[\lambda_{cwl}] = R[\lambda_{cwl}]/\mathrm{SNR}[\lambda_{cwl}]$

First, let's draw a single noisy entry:

[40]:
```
noise_obs = obs.add_noise(1, seed=0)
```

[41]:
```
axes = obs.plot_profiles(scope='illite', groupby='Sample ID', stacked=False,␣
↪hires_under=True, ci=False)
```

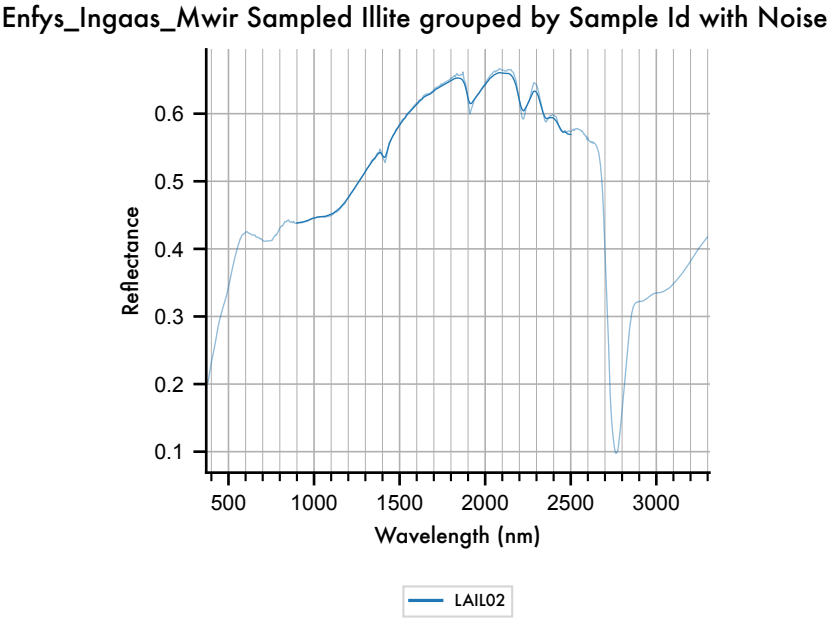


Figure 10 Noisy resampled example spectrum for Illite, showing 1 noisy sample according to the spectral signal-to-noise ratio illustrated in figure 3.B.

We can also plot the mean observed line, with shading indicating the standard deviation of the noisy data, shown below in figure 7, where the 'ci' keywrod has been applied ('ci': confidence-interval).

[42]:
```
axes = obs.plot_profiles(scope='illite', groupby='Sample ID', stacked=False,␣
↪hires_under=True, ci=True)
```

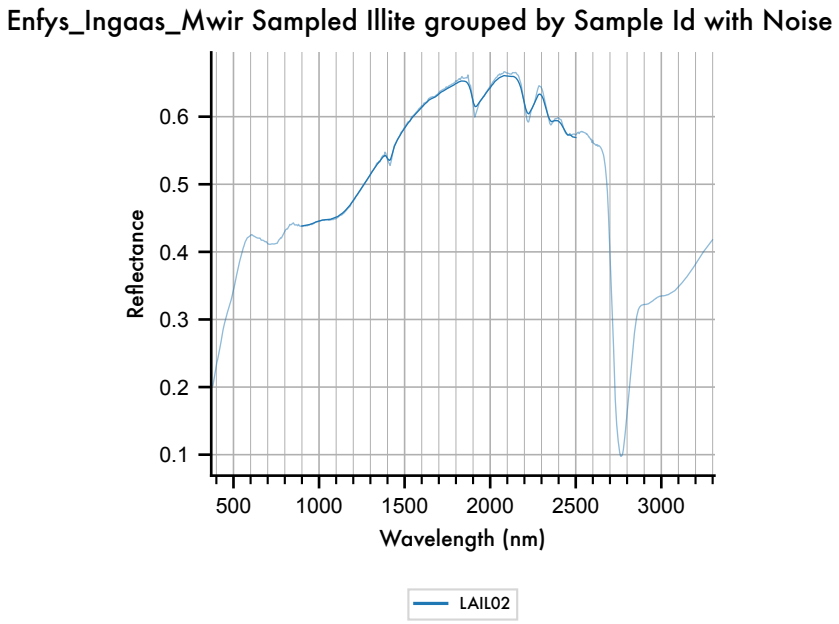


Figure 11 Mean of Noisy resampled example spectrum for Illite, showing mean and standard deviation of 100 repeat samples according to the spectral signal-to-noise ratio illustrated in figure 3.B.

Here we find that the standard deviation is almost negligible at the scale of the figure.

Plotting out all of the groups, we again see that the noise, at this scale, is indeed negligible for most entries. Here we opt to plot the complete range of noisy samples, rather than averaging and showing the (very small) standard deviation.

[43]:
```
axes = obs.plot_profiles(scope='categories', groupby='Species', stacked=True,␣
↪hires_under=True, ci=False)
```

```
Negative level values detected
Negative level values detected
```

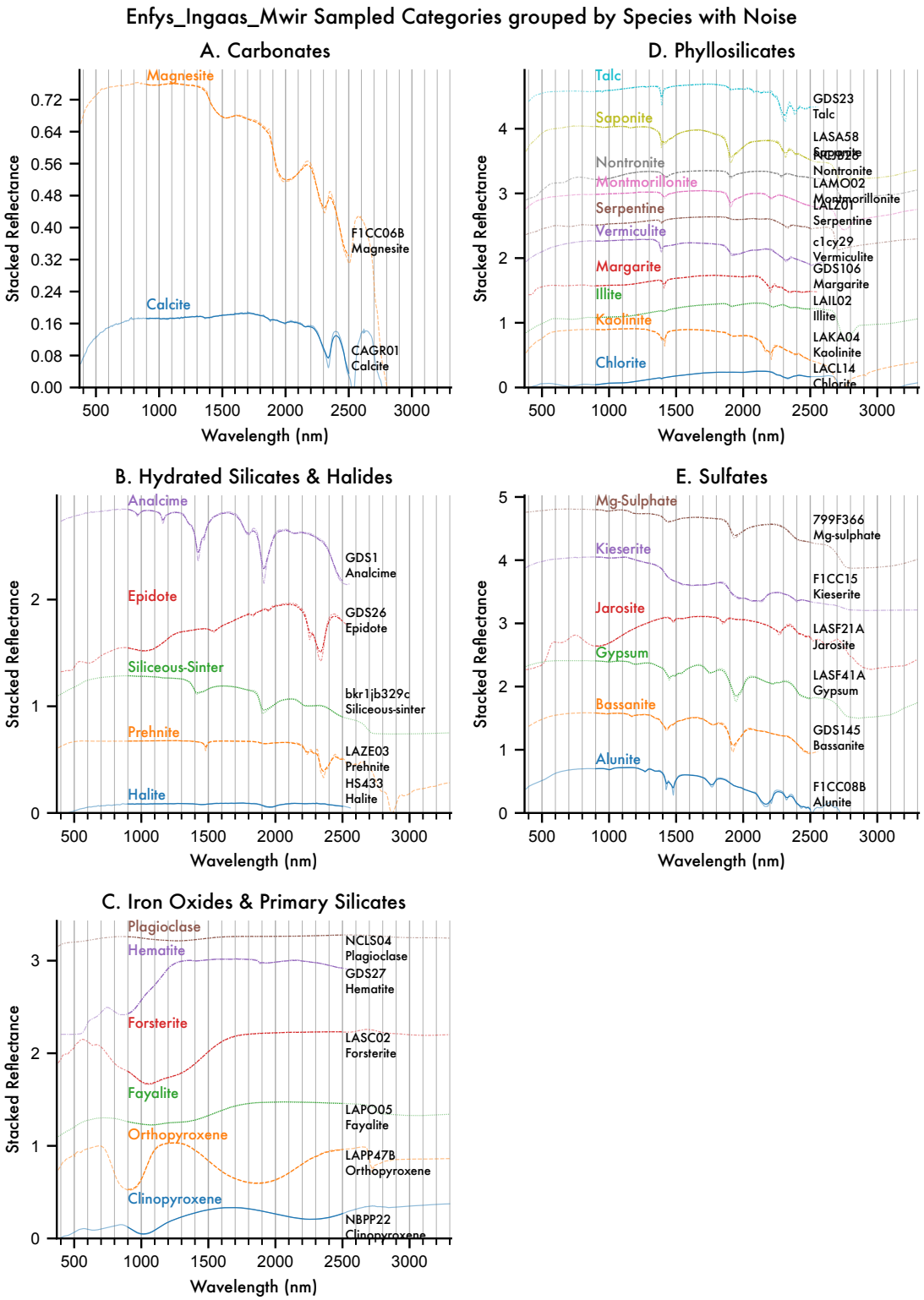


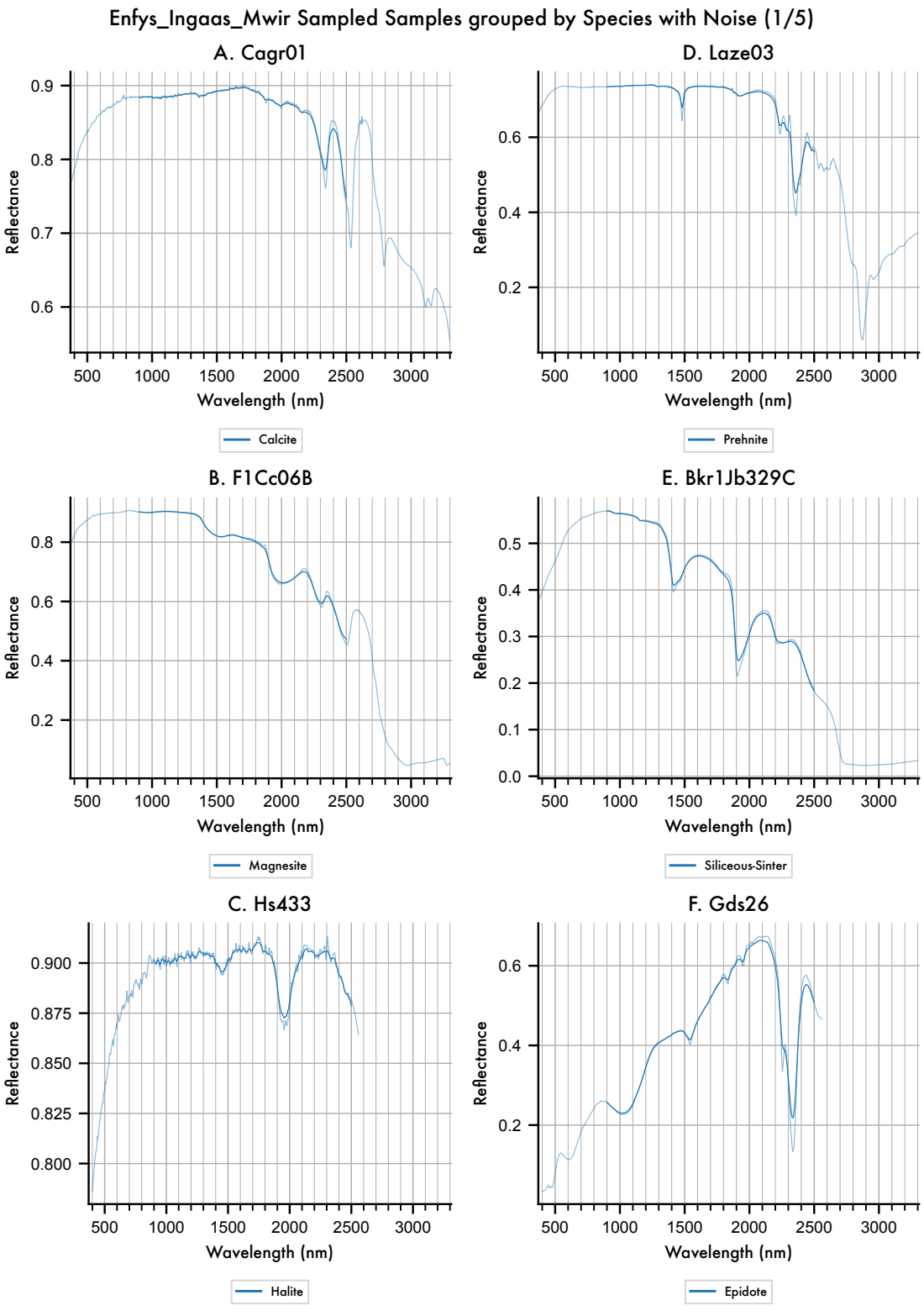
Figure 12 The MICA files as sampled by *Enfys*, with repeat samples drawn according to the ex-

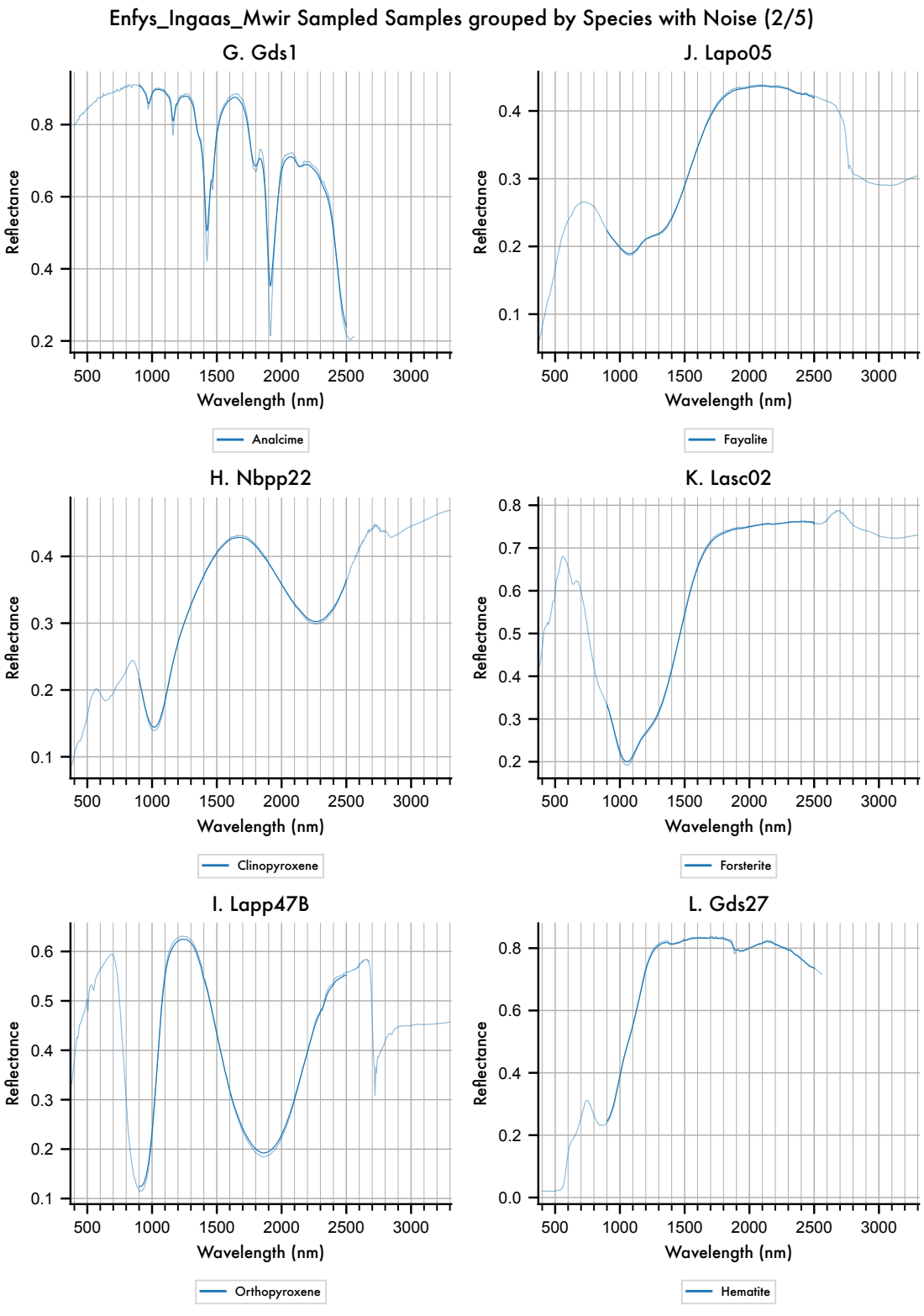
pected spectral Signal-to-Noise Ratio, with the original high-resolution spectra plotted underneath each.

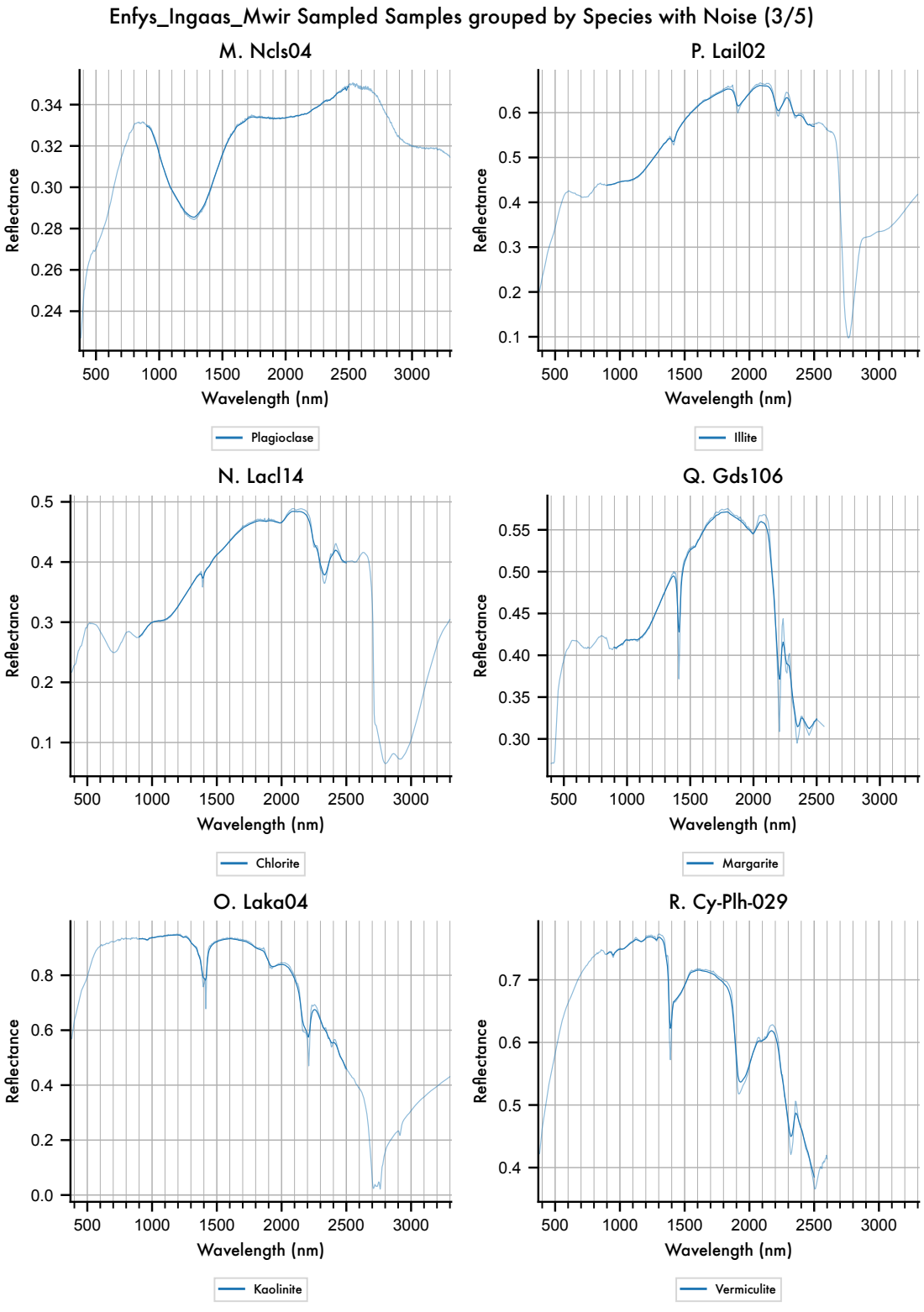
The key conclusion to draw from the above figure is that expected noise for *Enfys* does not significantly intrude on the diagnostic features of these type specimen of minerals identified on the Mars surface.

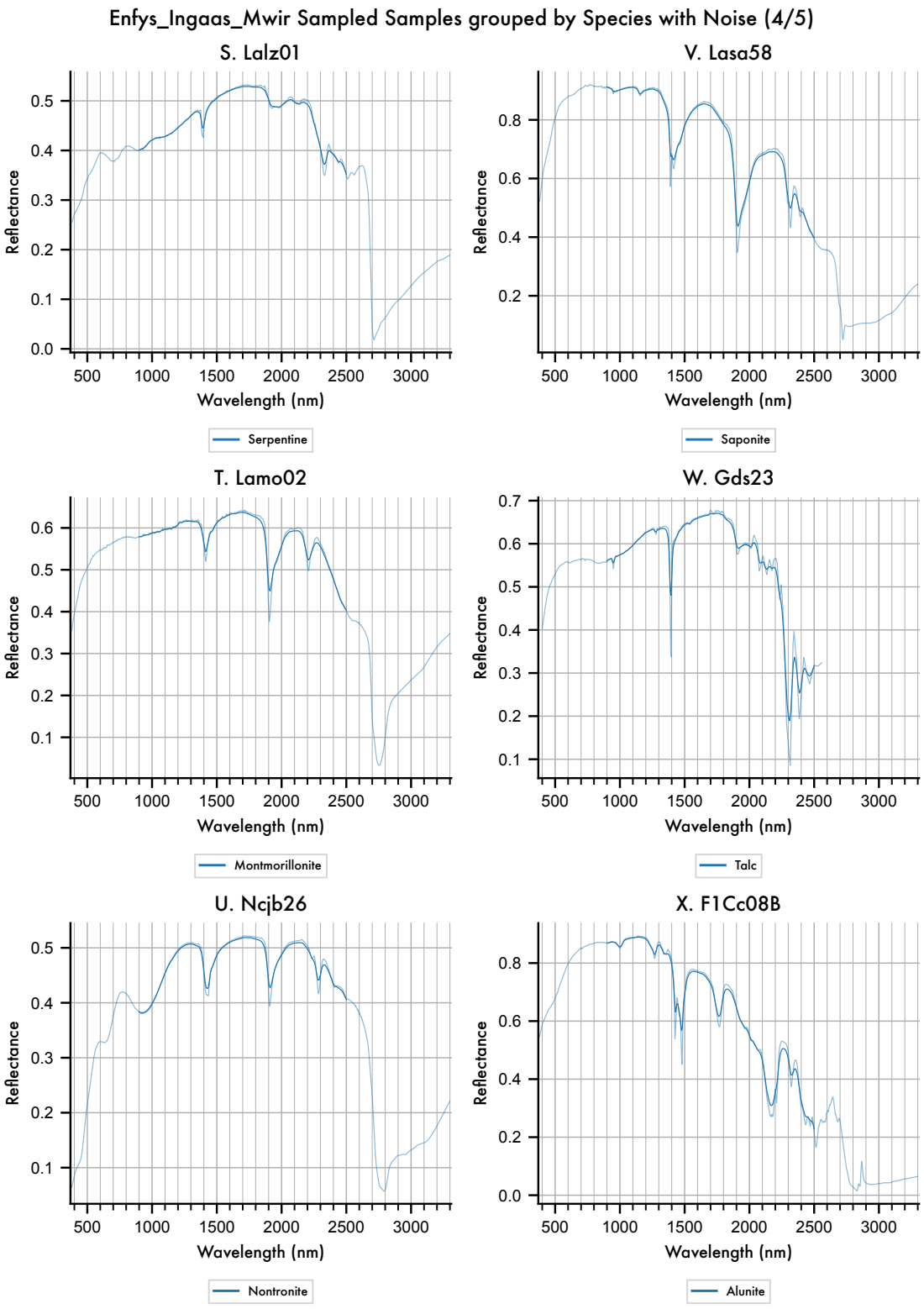
We can illustrate this in greater deal by plotting each entry on a separate figure, with the y-axis clipped to the spectral range of entry. Note that for sample with a small spectral range (i.e. samples that are spectrally flat), this stretching will appear to amplify noise, so the y-axis range must be considered when assessing the noise.

[44]:
```
axes = obs.plot_profiles(scope='samples', groupby='Species', stacked=False,␣
↪hires_under=True, ci=False)
```


Enfys_Ingaas_Mwir Sampled Samples grouped by Species with Noise (1/5)
A. Cagr01
D. Laze03
B. F1Cc06B
E. Bkr1Jb329C
C. Hs433
F. Gds26
Reflectance
Wavelength (nm)
Calcite
Prehnite
Magnesite
Siliceous-Sinter
Halite
Epidote


Enfys_Ingaas_Mwir Sampled Samples grouped by Species with Noise (2/5)
G. Gds1
J. Lapo05
H. Nbpp22
K. Lasc02
I. Lapp47B
L. Gds27
Reflectance
Wavelength (nm)
Analcime
Fayalite
Clinopyroxene
Forsterite
Orthopyroxene
Hematite


Enfys_Ingaas_Mwir Sampled Samples grouped by Species with Noise (3/5)
M. Ncls04
P. Lail02
N. Lacl14
Q. Gds106
O. Laka04
R. Cy-Plh-029
Reflectance
Wavelength (nm)
Plagioclase
Illite
Chlorite
Margarite
Kaolinite
Vermiculite


Enfys_Ingaas_Mwir Sampled Samples grouped by Species with Noise (4/5)
S. Lalz01
V. Lasa58
T. Lamo02
W. Gds23
U. Ncjb26
X. F1Cc08B
Reflectance
Wavelength (nm)
Serpentine
Saponite
Montmorillonite
Talc
Nontronite
Alunite

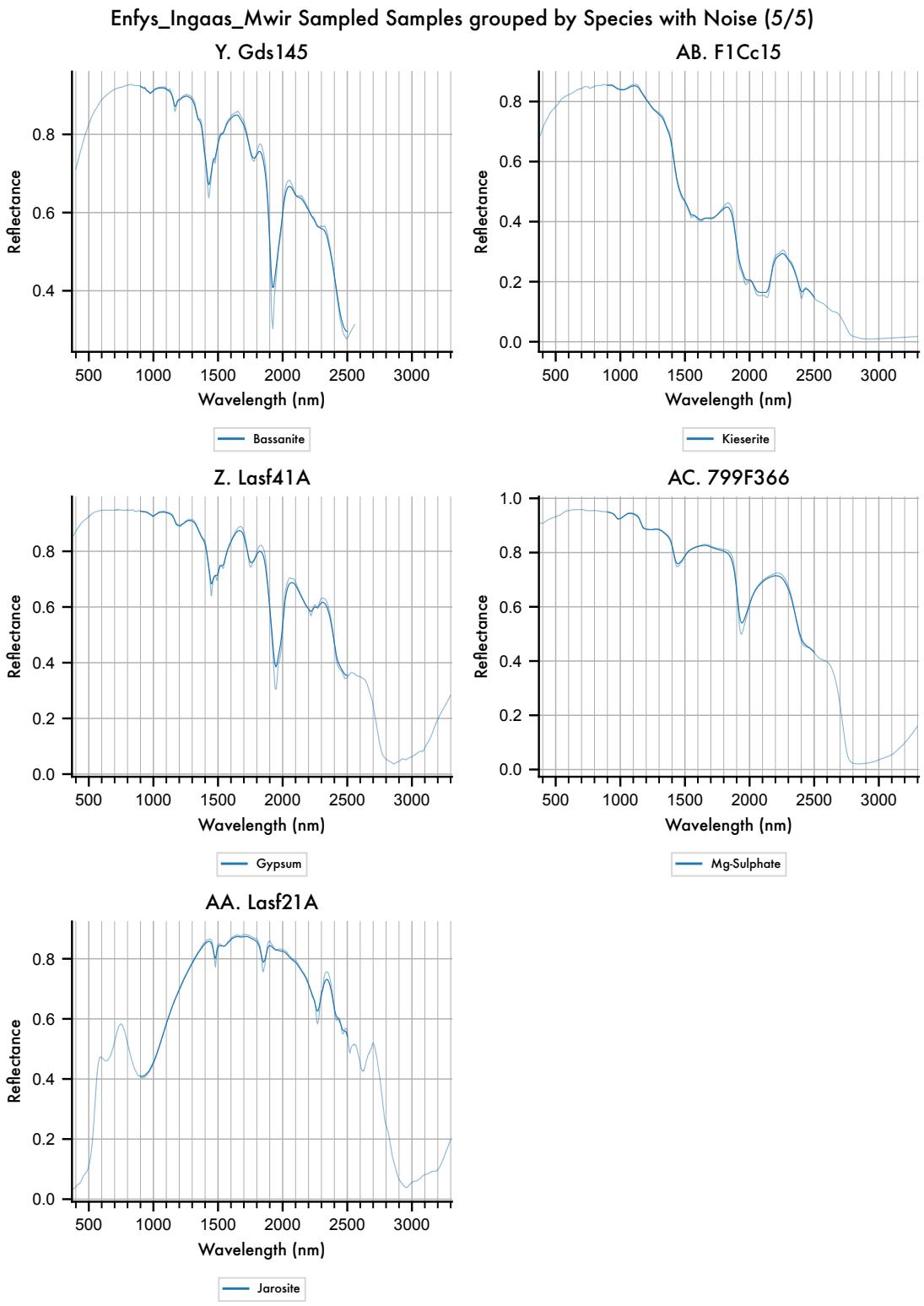


Figure 13 Detailled view of each entry in the spectral library, showing the scale of noise introduced,

relative to the features of each spectrum, and the reflectance range.

We can export the dataset:

[45]:
```
obs.export_main_df()
```

```
Exporting the Observation Pickle format…
Observation export complete.
```

5.4 Continuum Removal

We can apply continuum removal, even to the noisy data.

[46]:
```
from sptk.spectral_library_analyser import SpectralLibraryAnalyser as sla

cr_tool = sla(obs)
cr_obs = cr_tool.remove_continuum()
```

[47]:
```
axes = cr_obs.plot_profiles(scope='categories', groupby='Species',␣
↪stacked=True, hires_under=False, ci=False)
```

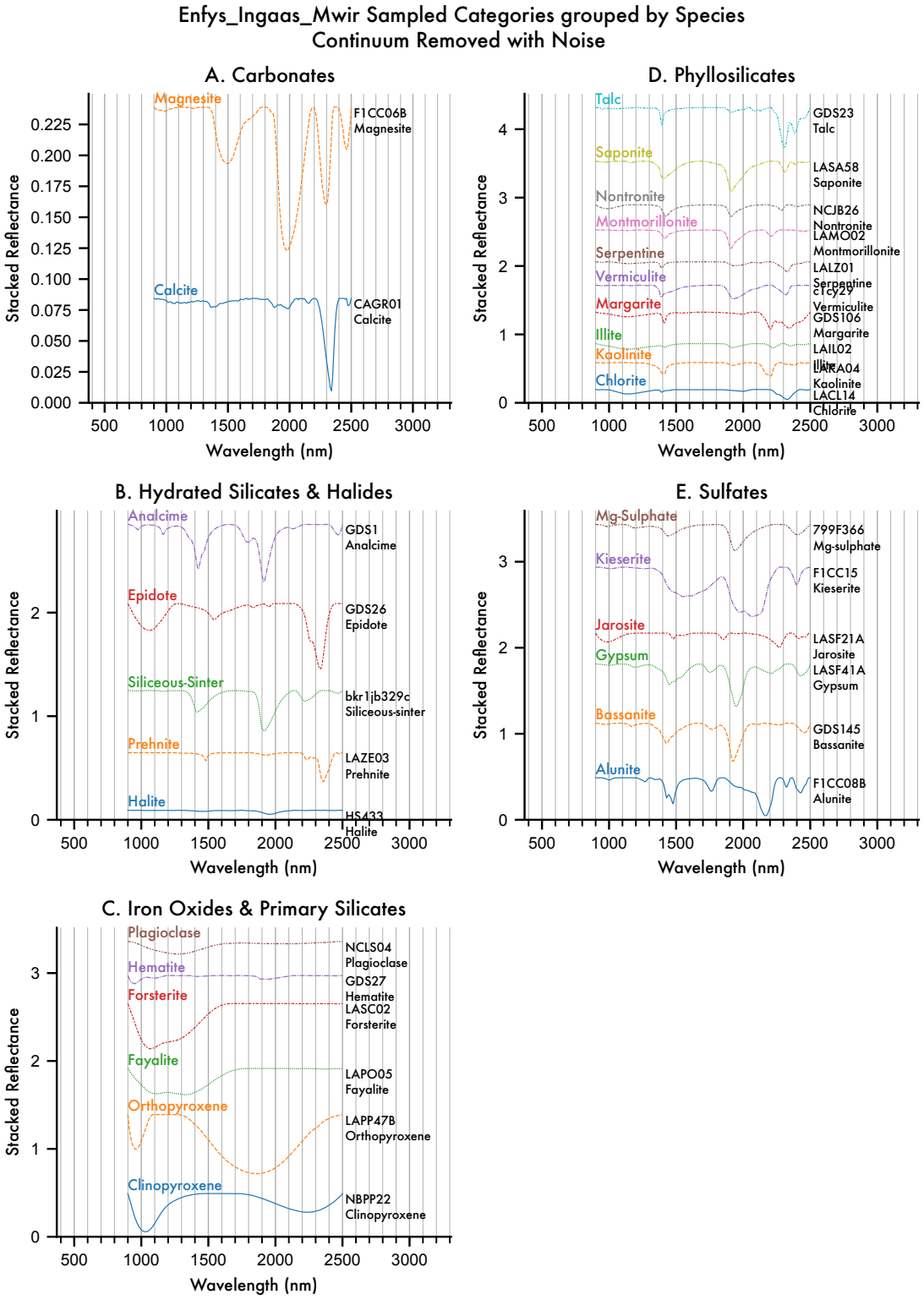


Figure 14 Continuum-Removed entries of the MICA Files, as sampled by *Enfys*, incldluing syn-

thetic noise.

Again, we can export this:

[48]:
```
cr_obs.export_main_df()
```

```
Exporting the Observation Pickle format…
Observation export complete.
```

5.5 Waterfall Visualisation

For waterfall visualisations, we have the advantage of being able to visualise the noise introduced for each of multiple entries in 2D.

Each bar of the visualisation i scomposed of a series of rows, with each row as a redrawn noisy sample.

For this, we redraw with 128 repeat samples.

[49]:
```
obs.add_noise(128, seed=0)
```

[49]:

Data ID	Category	Root Data ID	Sample ID	…
CAGR01 Calcite.	carbonates	CAGR01 Calcite	CAGR01	
CAGR01 Calcite.1	carbonates	CAGR01 Calcite	CAGR01	
CAGR01 Calcite.10	carbonates	CAGR01 Calcite	CAGR01	
CAGR01 Calcite.100	carbonates	CAGR01 Calcite	CAGR01	
CAGR01 Calcite.101	carbonates	CAGR01 Calcite	CAGR01	
…	…	…	…	
799F366 Mg-sulphate.95	sulfates	799F366 Mg-sulphate	799F366	
799F366 Mg-sulphate.96	sulfates	799F366 Mg-sulphate	799F366	
799F366 Mg-sulphate.97	sulfates	799F366 Mg-sulphate	799F366	
799F366 Mg-sulphate.98	sulfates	799F366 Mg-sulphate	799F366	
799F366 Mg-sulphate.99	sulfates	799F366 Mg-sulphate	799F366	

Data ID	Species	Subgroup	Group
CAGR01 Calcite.	calcite	calcites	carbonates
CAGR01 Calcite.1	calcite	calcites	carbonates
CAGR01 Calcite.10	calcite	calcites	carbonates
CAGR01 Calcite.100	calcite	calcites	carbonates
CAGR01 Calcite.101	calcite	calcites	carbonates
…	…	…	…
799F366 Mg-sulphate.95	mg-sulphate	hydrous_sulphates	sulphates
799F366 Mg-sulphate.96	mg-sulphate	hydrous_sulphates	sulphates
799F366 Mg-sulphate.97	mg-sulphate	hydrous_sulphates	sulphates
799F366 Mg-sulphate.98	mg-sulphate	hydrous_sulphates	sulphates
799F366 Mg-sulphate.99	mg-sulphate	hydrous_sulphates	sulphates

Data ID	Library Sample Description	Date Added	\

```
Data ID
CAGR01 Calcite.         MICA_new_dirtree                NaN        NaN
CAGR01 Calcite.1        MICA_new_dirtree                NaN        NaN
CAGR01 Calcite.10       MICA_new_dirtree                NaN        NaN
CAGR01 Calcite.100      MICA_new_dirtree                NaN        NaN
CAGR01 Calcite.101      MICA_new_dirtree                NaN        NaN
…                                  …                  …        …
799F366 Mg-sulphate.95  MICA_new_dirtree                NaN        NaN
799F366 Mg-sulphate.96  MICA_new_dirtree                NaN        NaN
799F366 Mg-sulphate.97  MICA_new_dirtree                NaN        NaN
799F366 Mg-sulphate.98  MICA_new_dirtree                NaN        NaN
799F366 Mg-sulphate.99  MICA_new_dirtree                NaN        NaN

                     Viewing Geometry  … 2414.54097557 2424.0627937  \
Data ID                                …
CAGR01 Calcite.                   NaN  …      0.838474     0.833697
CAGR01 Calcite.1                  NaN  …      0.838423     0.834497
CAGR01 Calcite.10                 NaN  …      0.837818     0.834064
CAGR01 Calcite.100                NaN  …      0.838397     0.833239
CAGR01 Calcite.101                NaN  …      0.838155     0.833639
…                                   …  …           …            …
799F366 Mg-sulphate.95            NaN  …      0.469208     0.463022
799F366 Mg-sulphate.96            NaN  …      0.469681     0.463516
799F366 Mg-sulphate.97            NaN  …      0.469822     0.463061
799F366 Mg-sulphate.98            NaN  …      0.469197     0.463608
799F366 Mg-sulphate.99            NaN  …      0.469636     0.462679

                     2433.58876046 2443.11884877 2452.65303176  \
Data ID
CAGR01 Calcite.           0.828243      0.818816      0.808897
CAGR01 Calcite.1          0.828270      0.818178      0.809766
CAGR01 Calcite.10         0.827498      0.819103      0.808354
CAGR01 Calcite.100        0.827555      0.819878      0.808788
CAGR01 Calcite.101        0.827732      0.818510      0.809729
…                            …             …             …
799F366 Mg-sulphate.95    0.458954      0.456807      0.452686
799F366 Mg-sulphate.96    0.458957      0.455761      0.452915
799F366 Mg-sulphate.97    0.458369      0.456064      0.452770
799F366 Mg-sulphate.98    0.459349      0.455822      0.453145
799F366 Mg-sulphate.99    0.459533      0.456109      0.452875

                     2462.19128285 2471.73357568  2481.27988414  \
Data ID
CAGR01 Calcite.           0.796534      0.782346       0.767380
CAGR01 Calcite.1          0.797038      0.781830       0.767405
CAGR01 Calcite.10         0.795277      0.780030       0.767796
CAGR01 Calcite.100        0.796668      0.780104       0.769582
```

```
CAGR01 Calcite.101          0.795986      0.782634       0.769502
…                                 …             …              …
799F366 Mg-sulphate.95      0.449539      0.445521       0.441514
799F366 Mg-sulphate.96      0.450723      0.445774       0.440301
799F366 Mg-sulphate.97      0.449284      0.445557       0.441591
799F366 Mg-sulphate.98      0.449676      0.446591       0.442252
799F366 Mg-sulphate.99      0.449112      0.446098       0.441478

                        2490.83018234  2500.38444467
Data ID
CAGR01 Calcite.              0.757593       0.748962
CAGR01 Calcite.1             0.757711       0.749908
CAGR01 Calcite.10            0.757171       0.749024
CAGR01 Calcite.100           0.759476       0.751268
CAGR01 Calcite.101           0.756959       0.747518
…                                 …              …
799F366 Mg-sulphate.95       0.437580       0.433064
799F366 Mg-sulphate.96       0.435993       0.435423
799F366 Mg-sulphate.97       0.436146       0.429929
799F366 Mg-sulphate.98       0.435952       0.432081
799F366 Mg-sulphate.99       0.438958       0.431011

[3712 rows x 254 columns]
```

[50]:
```
axes = obs.plot_profiles(scope='all', groupby='Category', stacked=False,␣
↪waterfall=True, ci=False)
```

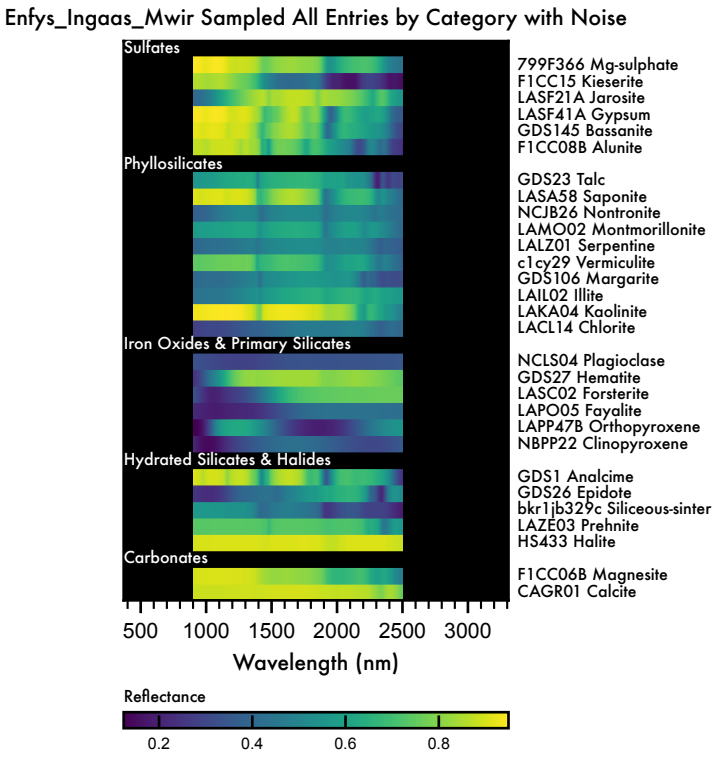


Figure 15 Waterfall diagram of the entries of the MICA Files, as sampled by *Enfys*, including synthetic noise. Each entry bar is composed of n repeat noisy samples, such that each bar is composed of n rows, illustrating the noise.

We need to recompute the continuum-removed data on this noisy set.

[51]:
```
cr_tool = sla(obs)
cr_obs = cr_tool.remove_continuum()
```

[52]:
```
axes = cr_obs.plot_profiles(scope='all', groupby='Category', stacked=False,␣
↪waterfall=True, ci=False)
```

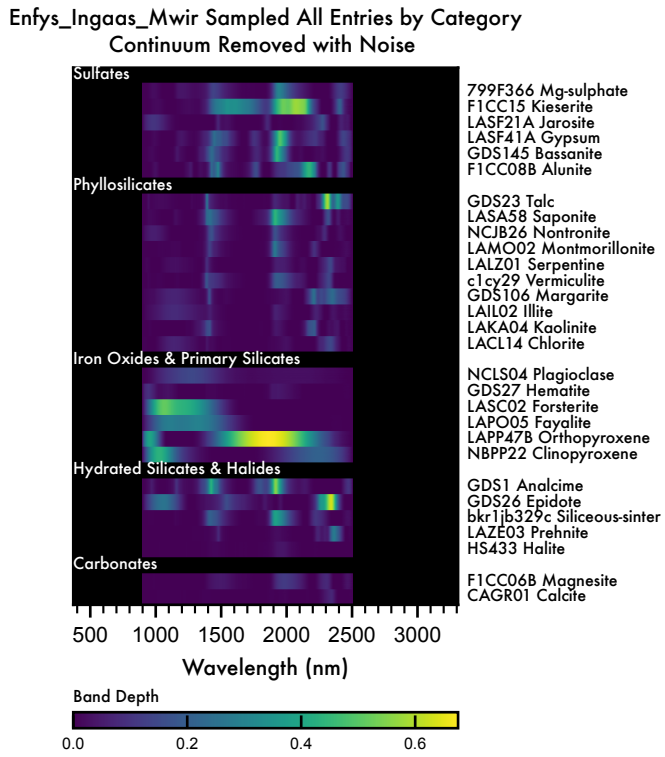
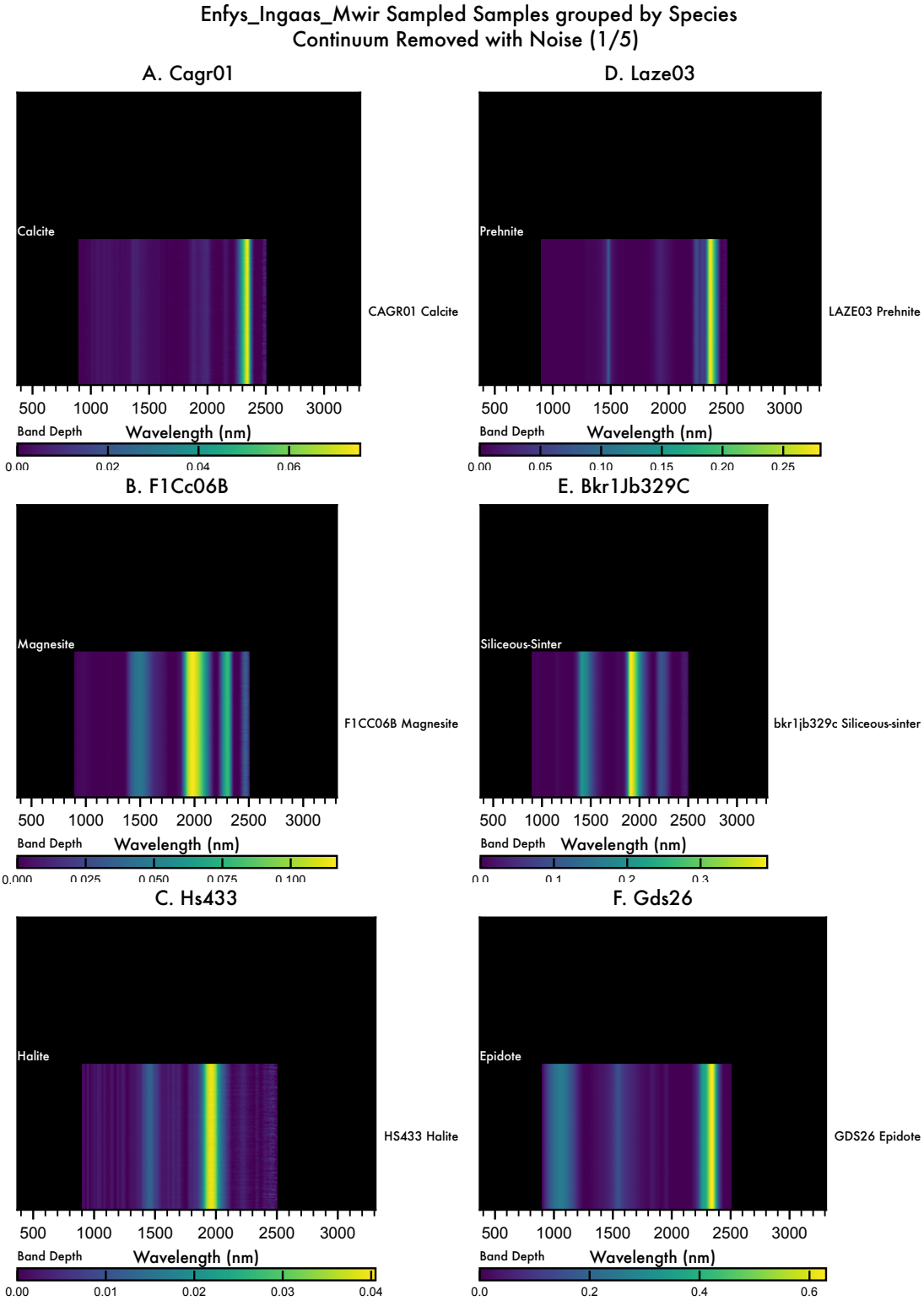


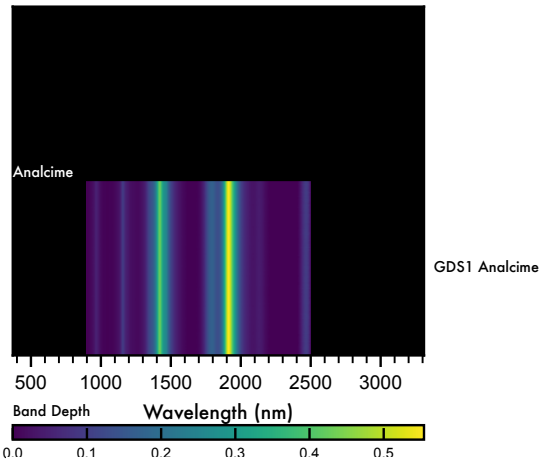
Figure 16 Continuum-Removed waterfall diagram of the entries of the MICA Files, as sampled by *Enfys*, including synthetic noise. Each entry bar is composed of n repeat noisy samples, such that each bar is composed of n rows, illustrating the noise.

[53]:
```
axes = cr_obs.plot_profiles(scope='samples', groupby='Species', stacked=False,␣
↪waterfall=True, ci=False)
```

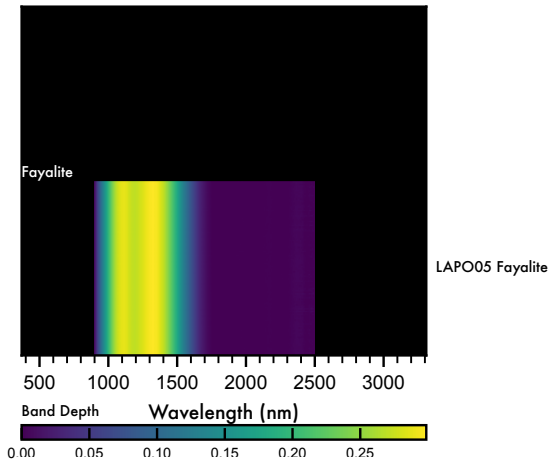

Enfys_Ingaas_Mwir Sampled Samples grouped by Species
Continuum Removed with Noise (1/5)
A. Cagr01
Calcite
CAGR01 Calcite
500 1000 1500 2000 2500 3000
Band Depth
Wavelength (nm)
0.00 0.02 0.04 0.06
D. Laze03
Prehnite
LAZE03 Prehnite
500 1000 1500 2000 2500 3000
Band Depth
Wavelength (nm)
0.00 0.05 0.10 0.15 0.20 0.25
B. F1Cc06B
Magnesite
F1CC06B Magnesite
500 1000 1500 2000 2500 3000
Band Depth
Wavelength (nm)
0.000 0.025 0.050 0.075 0.100
E. Bkr1Jb329C
Siliceous-Sinter
bkr1jb329c Siliceous-sinter
500 1000 1500 2000 2500 3000
Band Depth
Wavelength (nm)
0.0 0.1 0.2 0.3
C. Hs433
Halite
HS433 Halite
500 1000 1500 2000 2500 3000
Band Depth
Wavelength (nm)
0.00 0.01 0.02 0.03 0.04
F. Gds26
Epidote
GDS26 Epidote
500 1000 1500 2000 2500 3000
Band Depth
Wavelength (nm)
0.0 0.2 0.4 0.6

Enfys_Ingaas_Mwir Sampled Samples grouped by Species
Continuum Removed with Noise (2/5)

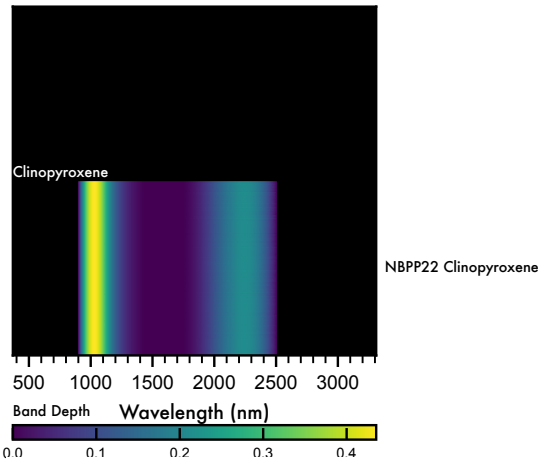
G. Gds1



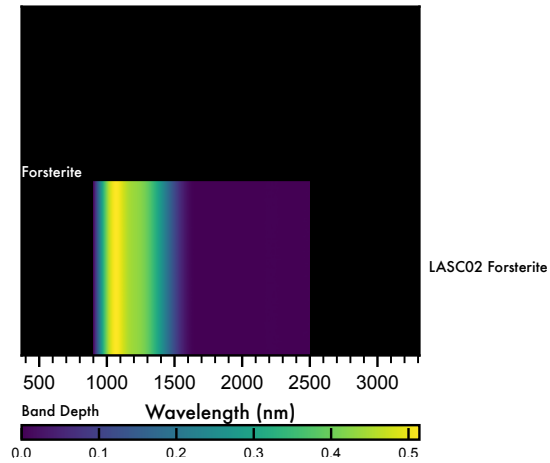
J. Lapo05



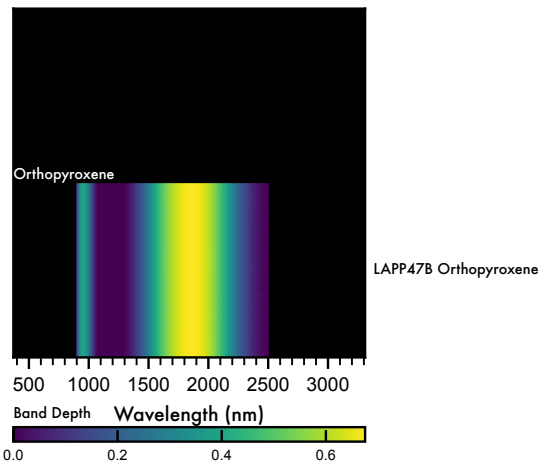
H. Nbpp22



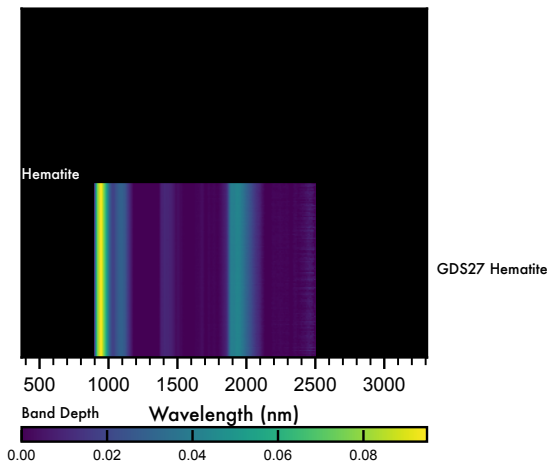
K. Lasc02



I. Lapp47B

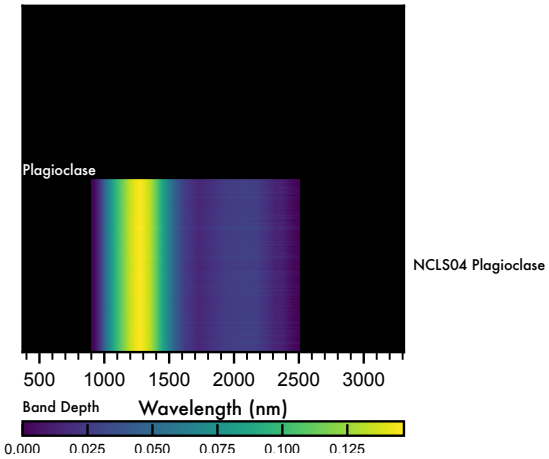


L. Gds27

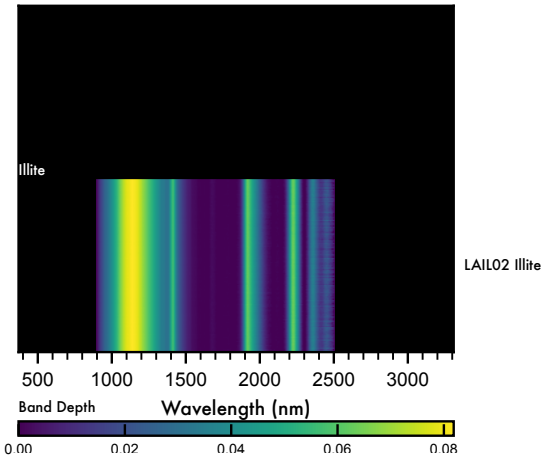


Enfys_Ingaas_Mwir Sampled Samples grouped by Species
Continuum Removed with Noise (3/5)

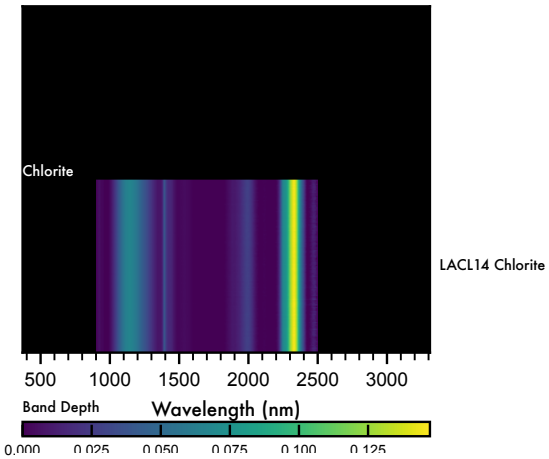
M. Ncls04



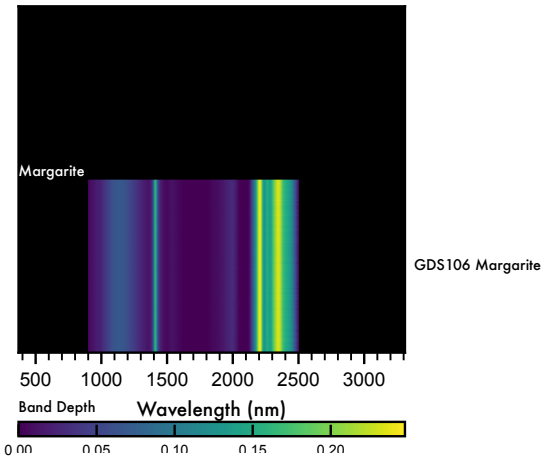
P. Lail02



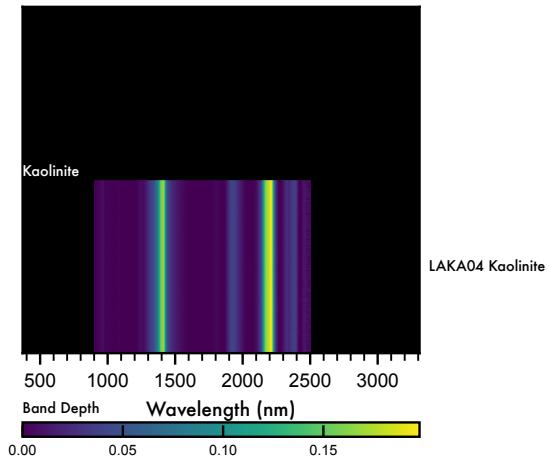
N. Lacl14



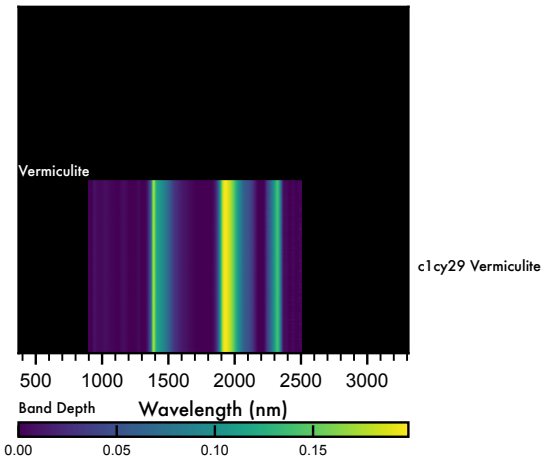
Q. Gds106



O. Laka04

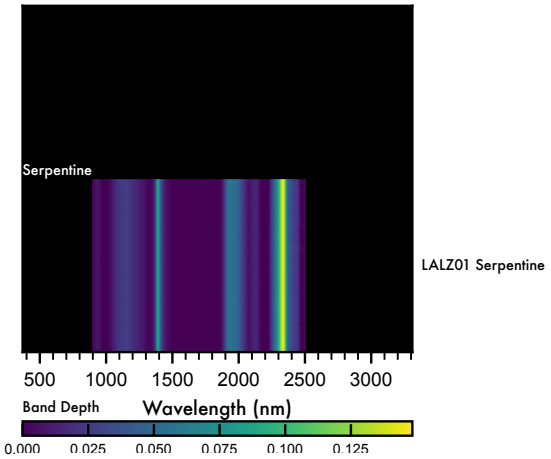


R. Cy-Plh-029

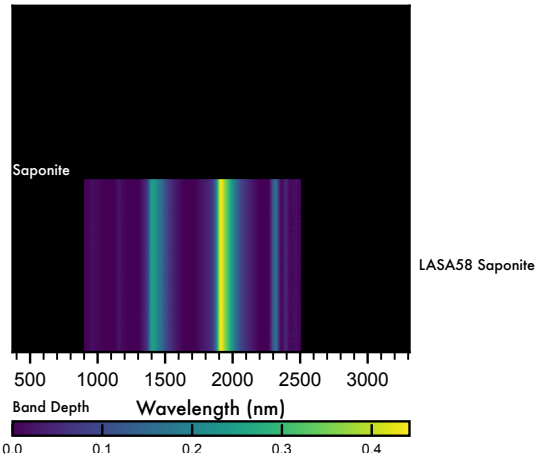


Enfys_Ingaas_Mwir Sampled Samples grouped by Species
Continuum Removed with Noise (4/5)

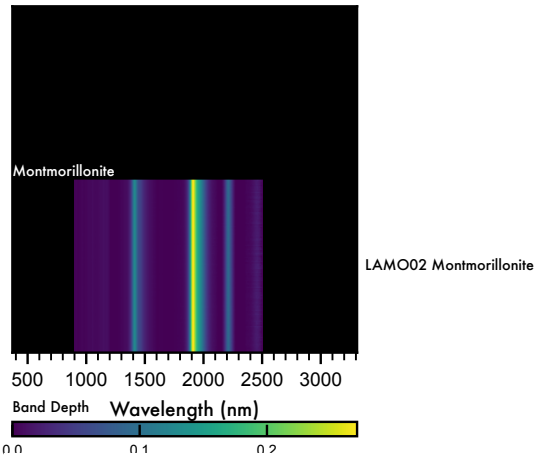
S. Lalz01



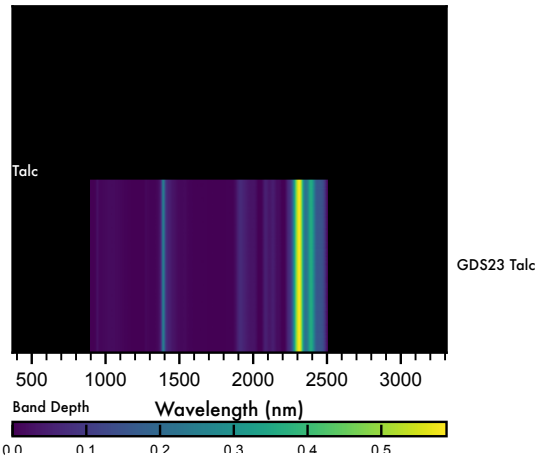
V. Lasa58



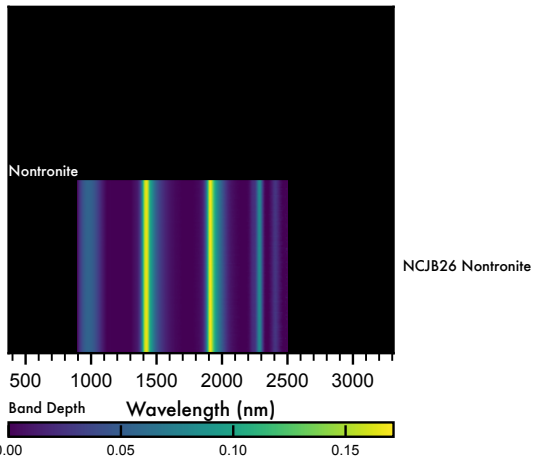
T. Lamo02



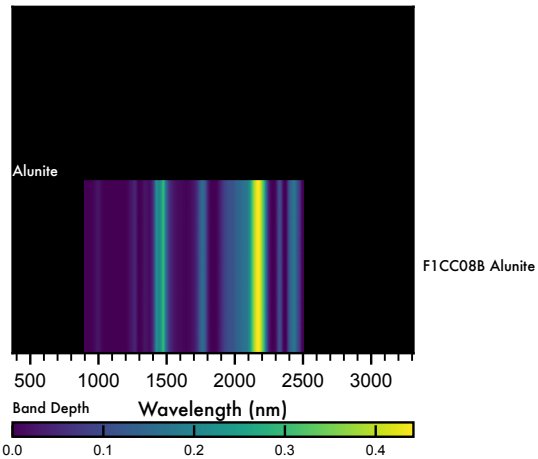
W. Gds23



U. Ncjb26



X. F1Cc08B



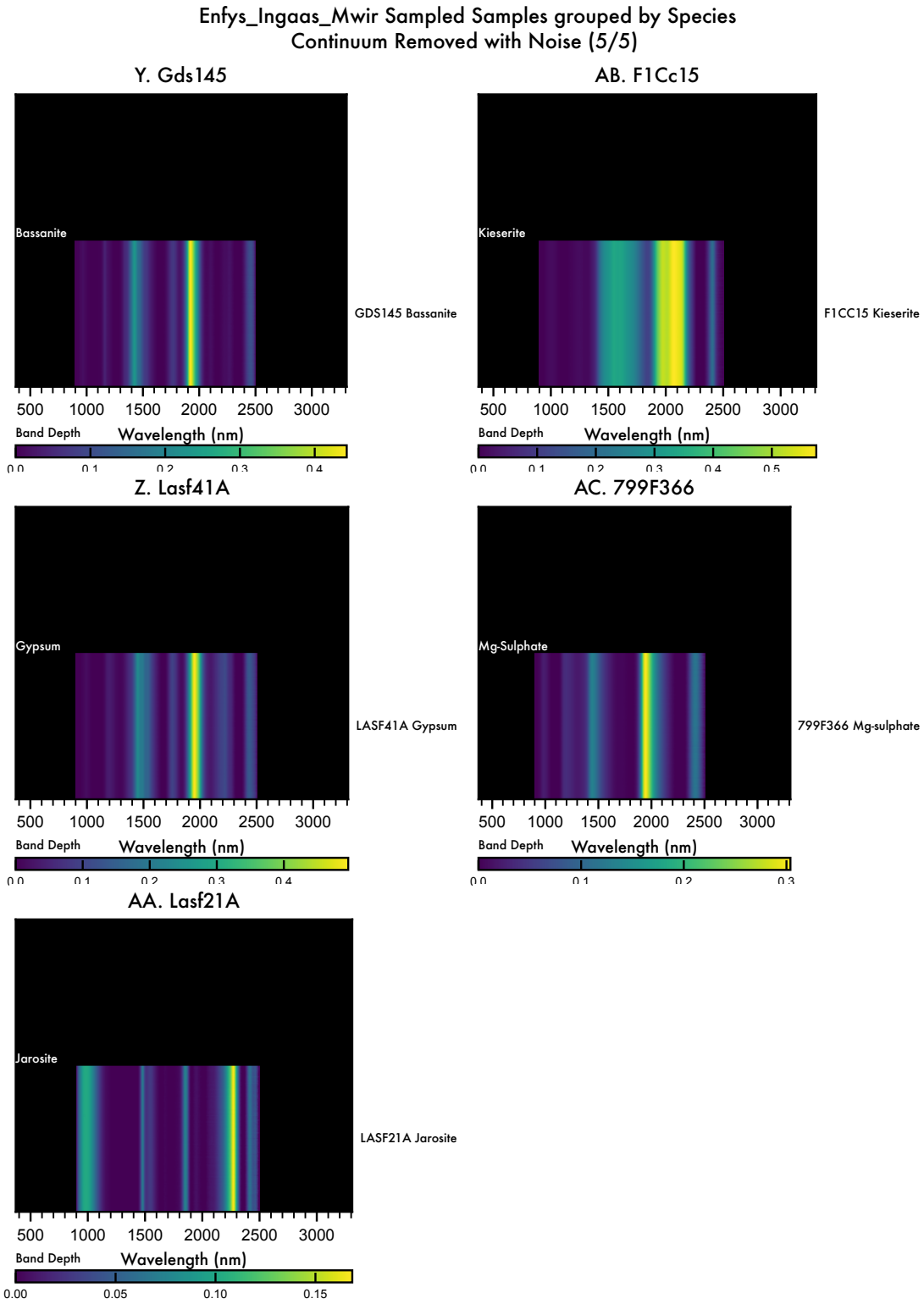


Figure 17 Detailed waterfall view of each entry in the spectral library, showing the scale of noise

introduced, relative to the features of each spectrum, and the reflectance range. Each entry bar is composed of n repeat noisy samples, such that each bar is composed of n rows, illustrating the noise.

5.5.1 Validating the Noise

We can check that our noise is correct by aggregating the repeat samples to give the mean and standard deviation for each entry and each spectral channel (wavelength).

We can then compare this to the spectral SNR.

[54]:
```
signal = obs.main_df.groupby('Root Data ID').mean()
noise = obs.main_df.groupby('Root Data ID').std()
snr = signal/noise
```

```
/var/folders/w4/44k32f413dd82lxmtw9456w80000gp/T/ipykernel_71187/2651352844.py:1
: FutureWarning: The default value of numeric_only in DataFrameGroupBy.mean is
deprecated. In a future version, numeric_only will default to False. Either
specify numeric_only or select only columns which should be valid for the
function.
  signal = obs.main_df.groupby('Root Data ID').mean()
/var/folders/w4/44k32f413dd82lxmtw9456w80000gp/T/ipykernel_71187/2651352844.py:2
: FutureWarning: The default value of numeric_only in DataFrameGroupBy.std is
deprecated. In a future version, numeric_only will default to False. Either
specify numeric_only or select only columns which should be valid for the
function.
  noise = obs.main_df.groupby('Root Data ID').std()
```

[55]:
```
snr.mean()
```

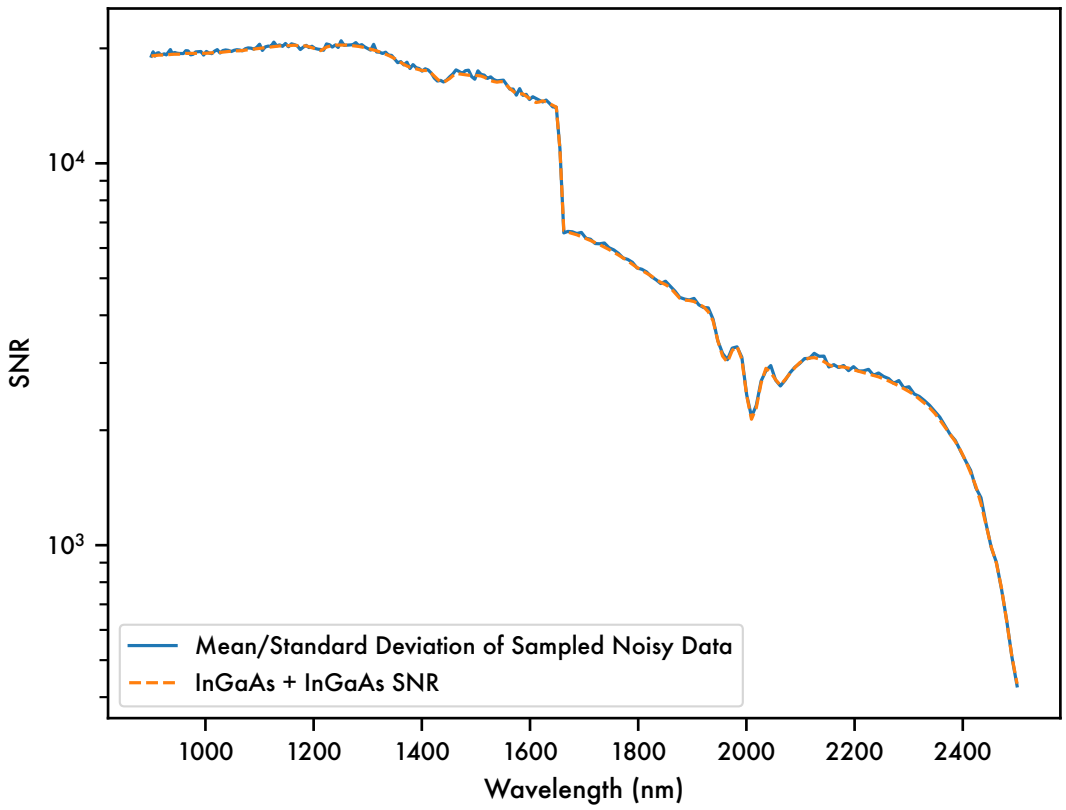
[55]:
```
900.000000     19079.373677
903.285743     19591.355181
906.618278     19084.958851
909.999291     19420.995764
913.430570     19486.295039
                   …
2462.191283      901.305491
2471.733576      764.822159
2481.279884      633.850178
2490.830182      509.898906
2500.384445      428.127727
Length: 237, dtype: float64
```

[60]:
```
# plot the SNR
import seaborn as sns
# plot with log scale y axis
g = sns.lineplot(data=snr.mean(), label='Mean/Standard Deviation of Sampled␣
 ↪Noisy Data', markers=True)
g.set(yscale="log")
```

```
# get instrument snr
sns.lineplot(data=obs.instrument.main_df, x='cwl', y='snr', label='InGaAs +␣
↪InGaAs SNR', linestyle='--', markers=True)

g.set_xlabel('Wavelength (nm)')
g.set_ylabel('SNR')
```

[60]: Text(0, 0.5, 'SNR')



5.6 Spectral Angle Mapper

TODO: perform spectral angle mapper evaluation of each spectrum against the other spectra of the set.

6 Discussion

Notes on what we found above, perhaps with some more analysis.

7 Conclusions

Concluding remarks and summary of findings.

8 References